MATURI VENKATA SUBBA RAO ENGINEERING COLLEGE

(An Autonomous Institution)

(Affiliated to Osmania University & Recognized by AICTE) Nadergul, Ranga Reddy Dist.



CERTIFICATE

DEPARTMENT OF COMPUTER SCIENCE &ENGINEERING

This is to certify that it is a bonafide laboratory work of experiments carried out by Mr. /Miss. _____bearing R.No. _____under the laboratory course **Parallel & Distributed Computing Lab (U21PC782CS)** prescribed for B.E **IV/IV** SEM-VII by Osmania University in the department during the academic year **2024-2025**.

Internal Examiner ***HOD*** ***External Examiner***

VISION AND MISSION

VISION

- To impart technical education of the highest standards, producing competent and confident engineers with an ability to use computer science knowledge to solve societal problems.

MISSION

- To make learning process exciting, stimulating and interesting.
- To impart adequate fundamental knowledge and soft skills to students.
- To expose students to advanced computer technologies in order to excel in engineering practices by bringing out the creativity in students.
- To develop economically feasible and socially acceptable software.

PROGRAM EDUCATIONAL OBJECTIVES (PEOs)

The Bachelor's program in Computer Science and Engineering is aimed at preparing graduates who will:-
PEO-1: Achieve recognition through demonstration of technical competence for successful execution of software projects to meet customer business objectives.
PEO-2: Practice life-long learning by pursuing professional certifications, higher education or research in the emerging areas of information processing and intelligent systems at a global level.
PEO-3: Contribute to society by understanding the impact of computing using a multidisciplinary and ethical approach.

(A) PROGRAM OUTCOMES(POs)

At the end of the program the students (Engineering Graduates) will be able to:

1. **Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
2. **Problem analysis:** Identify, formulate, review research literature, and analyses complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
3. **Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
4. **Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
5. **Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
6. **The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
7. **Environment and sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
8. **Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
9. **Individual and team work:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
10. **Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
11. **Project management and finance:** Demonstrate knowledge and understanding of the engineering and management principle and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
12. **Lifelong learning:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

(B) PROGRAM SPECIFIC OUTCOMES(PSOs)

13. (PSO-1) Demonstrate competence to build effective solutions for computational real-world problems using software and hardware across multi-disciplinary domains.
14. (PSO-2) Adapt to current computing trends for meeting the industrial and societal needs through holistic professional development leading to pioneering careers or entrepreneurship

Course Name: Parallel & Distributed Computing Lab

Course Code: U21PC782CS

Academic Year: 2024-2025

Semester: VII

Course Objectives

 To implement client and server programs using sockets

 To learn about working of NFS

 Understanding Remote Communication and Interprocess Communication

 To use Map, reduce model for distributed processing

 To develop mobile applications

Course Outcomes

After completing this course, the student will be able to

 Write programs that communicate data between two hosts

 Configure NFS

 To implement inter process communication and remote communication

 Use distributed data processing frameworks and mobile application tool kits

INDEX

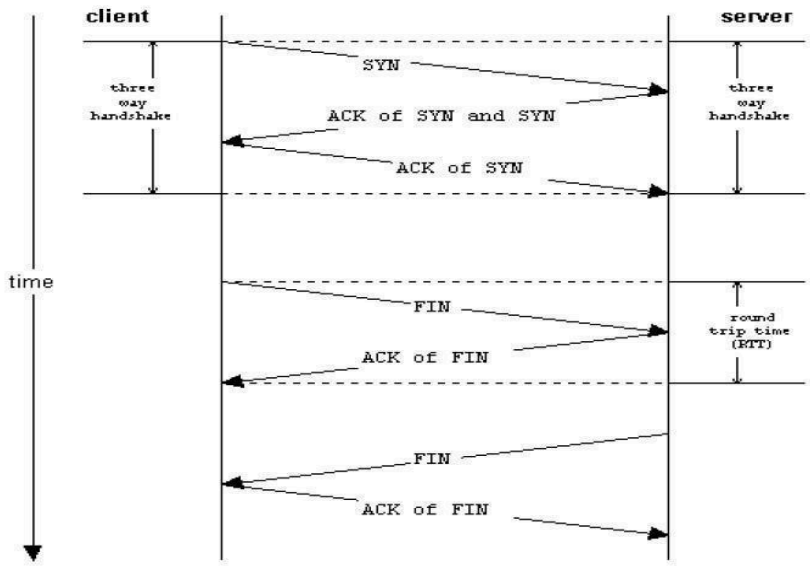
S.No.	Name of the Experiment	Experiment Date	Date of Submission	Page No.

1. Implementing Client-Server Communication using TCP

Introduction:

TCP provides the service of exchanging data directly between two network hosts, whereas IP handles addressing and routing message across one or more networks. In particular, TCP provides reliable, ordered delivery of a stream of bytes from a program on one computer to another program on another computer.

TCP is the protocol that major Internet applications rely on, applications such as the World Wide Web, email, and file transfer. TCP connection is managed by an operating system through a programming interface that represents the local end-point for communications, the *Internet socket*. During the lifetime of a TCP connection it undergoes a series of state changes:



1. LISTEN: In case of a server, waiting for a connection request from any remote client.
2. SYN-SENT: waiting for the remote peer to send back a TCP segment with the SYN and ACK flags set. (Usually set by TCP clients)
3. SYN-RECEIVED: waiting for the remote peer to send back an acknowledgment after having sent back a connection acknowledgment to the remote peer. (usually set by TCP servers) 4. ESTABLISHED: the port is ready to receive/send data from/to the remote peer.
5. FIN-WAIT-1
6. FIN-WAIT-2
7. CLOSE-WAIT
8. CLOSING
9. LAST-ACK
10. TIME-WAIT: represents waiting for enough time to pass to be sure the remote peer received the acknowledgment of its connection termination request. According to RFC 793 a connection can

stay in TIME-WAIT for a maximum of four minutes.

11. CLOSED

TCP uses the notion of port numbers to identify sending and receiving application end-points on a host, or *Internet sockets*. Each side of a TCP connection has an associated 16-bit unsigned port number (0-65535) reserved by the sending or receiving application. Arriving TCP data packets are identified as belonging to a specific TCP connection by its sockets, that is, **the combination of source host address, source port, destination host address, and destination port**. This means that a server computer can provide several clientswith several services simultaneously, as long as a client takes care of initiating any simultaneous connections toone destination port from different source ports.

TCPServer

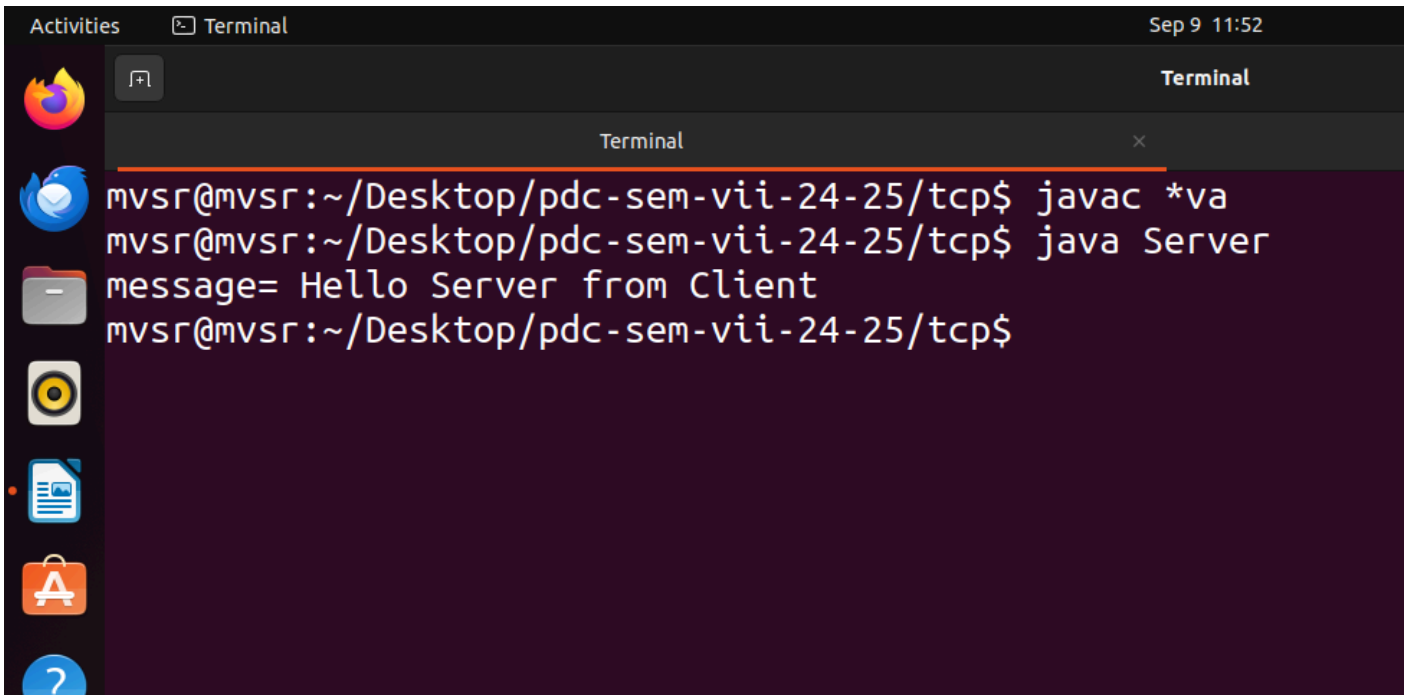
```
import java.io.*;
import java.net.*;
public class TCPServer {
public static void main(String[] args)
{
try{
ServerSocket ss=new
ServerSocket(8034);
Socket s=ss.accept();//establishes
connection
DataInputStream dis=new
DataInputStream(s.getInputStream
());
String
str=(String)dis.readUTF(
);
System.out.println("message= "+str);
ss.close();
}
catch(Exception e)
{
System.out.println(e);
}
}
}
```

TCPClient.java

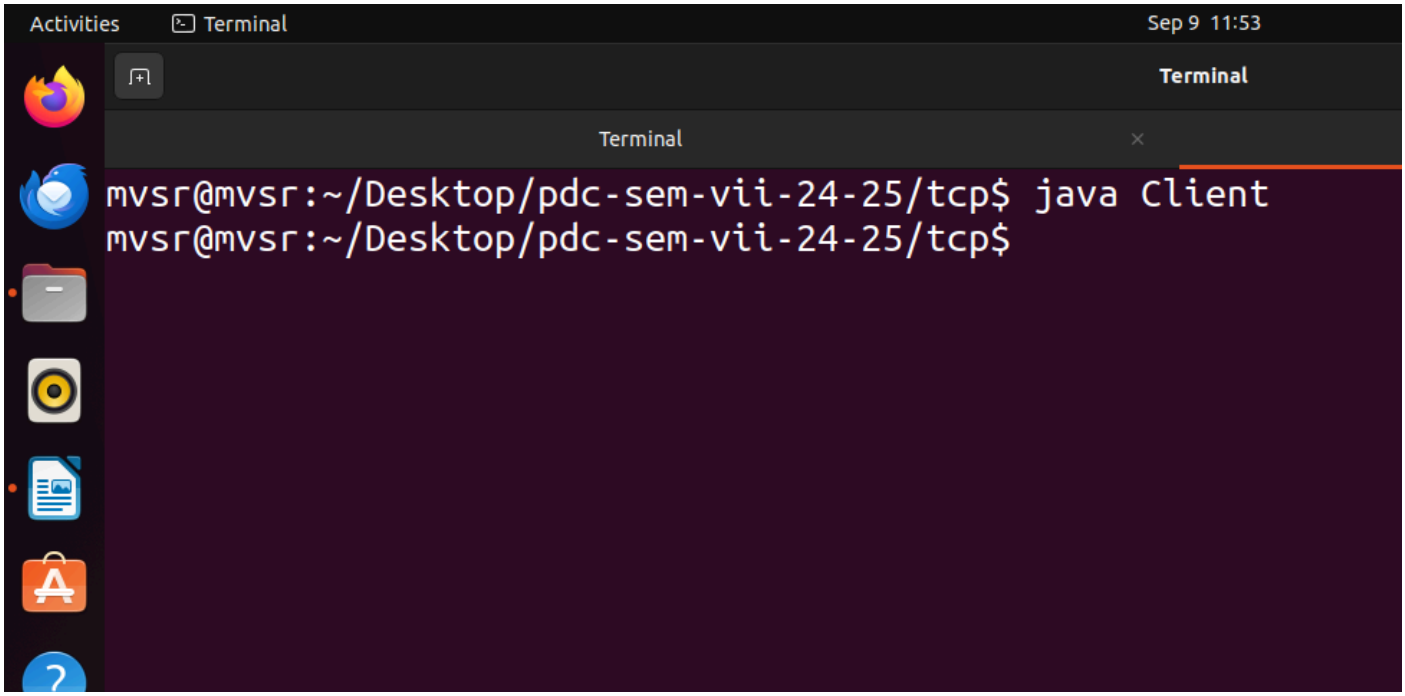
```
import java.io.*;
import java.net.*;
public class Client {
public static void main(String[] args) {
try{
Socket s=new Socket("127.0.0.1",8034);
DataOutputStream dout=new DataOutputStream(s.getOutputStream());
dout.writeUTF("Hello Server from Client");
dout.flush();
dout.close();
s.close();
}
catch(Exception e)
{System.out.println(e);
}
}
}
```

Sample Outputs:

TCPServer



TCPClient



2.Implementation of client-server communication using UDP

Introduction:

With UDP, computer applications can send messages, in this case referred to as *data-grams*, to other hosts on an Internet Protocol (IP) network without requiring prior communications to set up special transmission channels or data paths. UDP uses a simple transmission model without implicit hand-shaking for providing reliability, ordering, or data integrity. UDP's stateless nature is also useful for servers answering small queries from huge numbers of clients. Unlike TCP, UDP is compatible with packet broadcast (sending to all on local network) and multicasting (send to all subscribers).Common network applications that use UDP include the Domain Name System (DNS).

UDP applications use **datagram sockets** to establish host-to-host communications. A **Datagram socket** is a type of Internet socket, which is the sending or receiving point for packet delivery services.[1] Each packet sent or received on a Datagram socket is individually addressed and routed. Multiple packets sent from one machine to another may arrive in any order and might not arrive at the receiving computer.UDP broadcasts sends are always enabled on a Datagram Socket. In order to receive broadcast packets a Data-gram Socket must be bound to a more specific address.

An application binds a socket to its endpoint of data transmission, which is a combination of an IP address and a service port. A port is a software structure that is identified by the port number, a 16 bit integer value, allowing for port numbers between 0 and 65535.

Port numbers are divided into three ranges:

- Port numbers 0 through 1023 are used for common, well-known services.
- Port numbers 1024 through 49151 are the registered ports
- Ports 49152 through 65535 are dynamic ports that are not officially used for any specific service, and can be used for any purpose. They are also used as ephemeral ports , from which software

running on the host may randomly choose a port in order to define itself.[2] In effect, they are used as temporary ports primarily by clients when communicating with servers.

UDPServer

```
import java.net.*;
public class UServer{
  public static void main(String[] args) throws Exception {
    DatagramSocket ds = new DatagramSocket(8057);
    byte[] buf = new byte[1024];
    DatagramPacket dp = new DatagramPacket(buf, 1024);
    ds.receive(dp);
    String str = new String(dp.getData(), 0, dp.getLength());
    System.out.println(str);
    ds.close();
  }
}
```

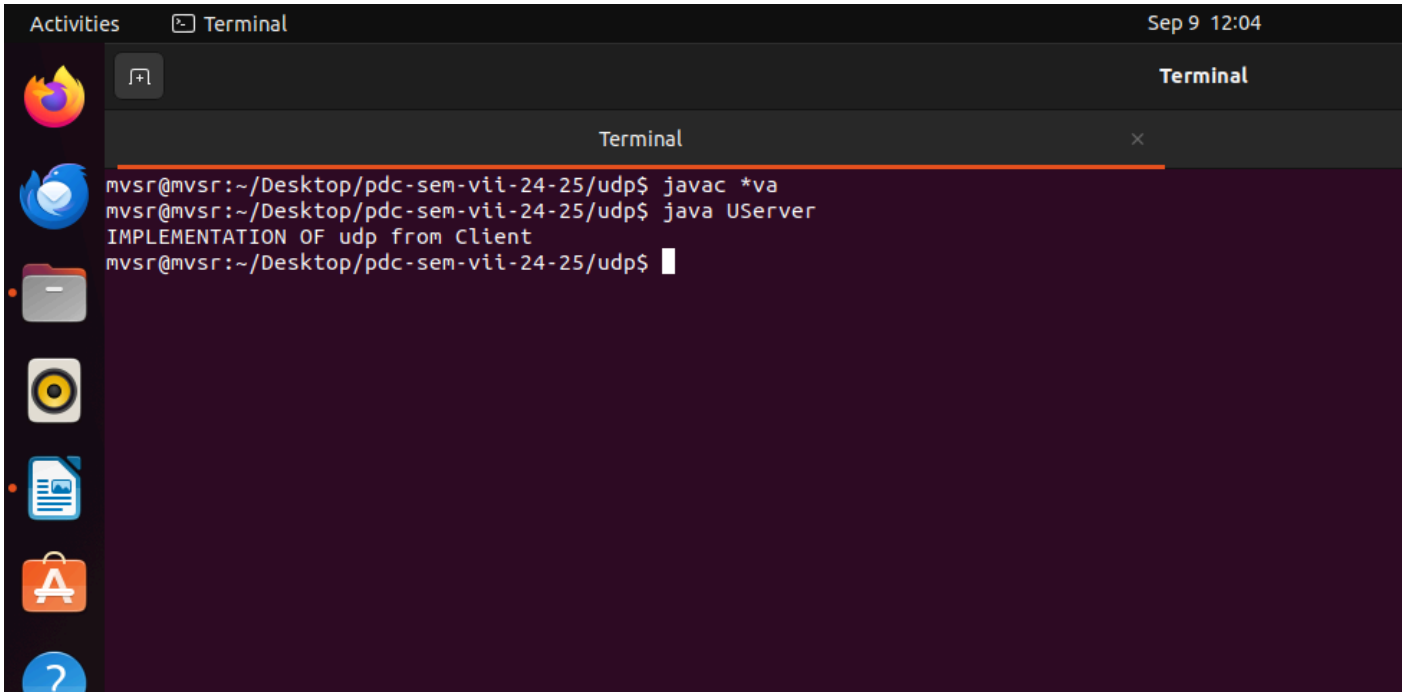
UDPClient.java

```
import java.net.*;
public class UClient{
  public static void main(String[] args) throws Exception {
    DatagramSocket ds = new DatagramSocket();
    String str = "IMPLEMENTATION OF udp from Client";
    InetAddress ip = InetAddress.getByName("127.0.0.1");

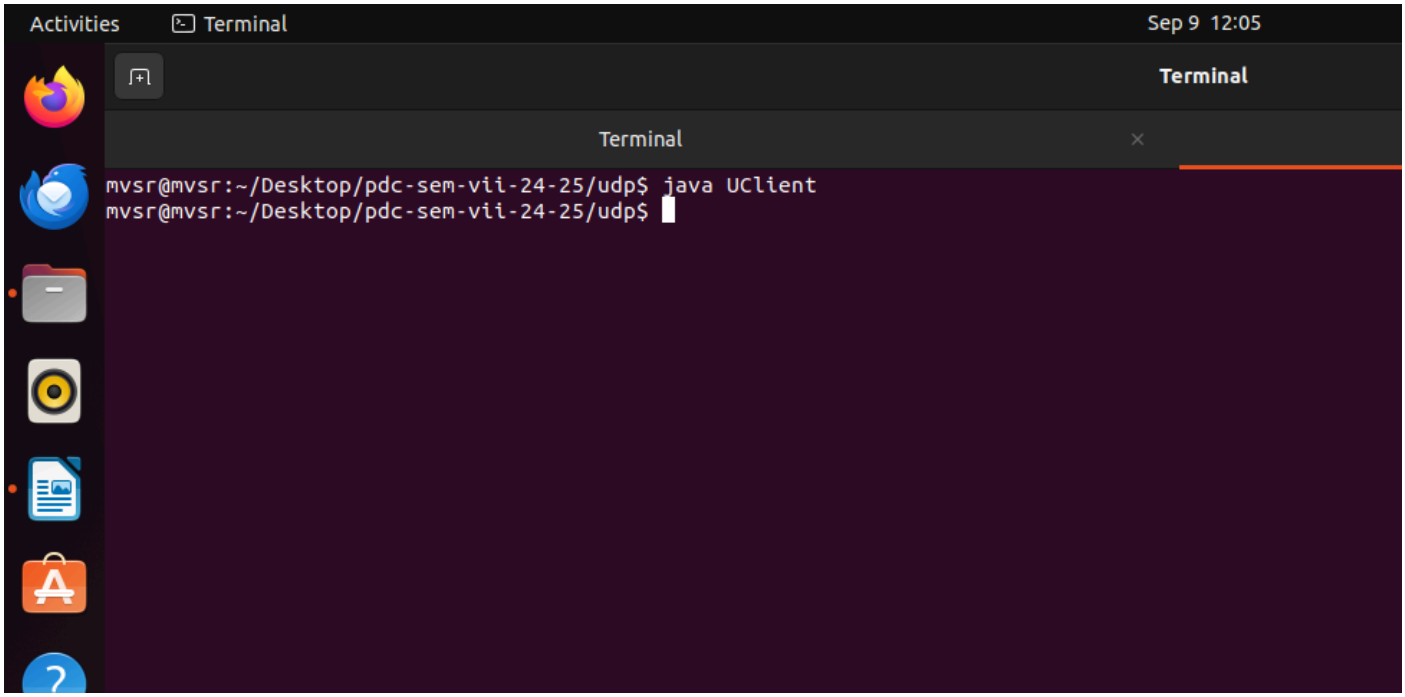
    DatagramPacket dp = new DatagramPacket(str.getBytes(), str.length(), ip, 8057);
    ds.send(dp);
    ds.close();
  }
}
```

Sample Outputs:

UDPServer



UDPClient



3.Chat Server Implementation

Introduction:

Chat server is a standalone application that is made of combination of two-applications, server application (which runs on server side) and client application (which runs on client side). This application is using for chatting in LAN. To start chatting client must be connected with the server

after which, messages can be broadcast between each and every client.

SERVER:

Server side application is used to get the message from any client and broadcast to each and every client. And this application is also used to maintain the list of users and broadcast this list to everyone. Firstly server program creates a new server socket by the

ServerSocket ss = new ServerSocket(Port number);

- After creating the ServerSocket it accepts the client socket and adds this socket into an arraylist.
- After getting the client socket it creates a thread and enables the DataInputStream for this socket.
- After creating the input stream it reads the user name and adds it to arraylist and this arraylist object writes into the ObjectOutputStream of each client by using an iterator.
- After this process, it creates a new thread and creates one DataInputStream for reading the messages which is sent by the client and after reading the message it creates the DataOutputStream for each socket and writes this message in each client output stream through the iterator. If any client logs out,the server receives the client name and removes it from the array list. Further ,it sends this updated arraylist to all clients.

CLIENT:

In the client side, first the program creates a new socket and specifies the address and port of the server and establishes the connection with the Server.The client, then, creates a new thread and DataInputStream, ObjectInputStream and DataOutputStream for sending the user name and retrieving the list of all users and adds all the user names into its list box through the iterator.Then new thread is created for sending and receiving the messages from the server. This task is done by using DataInputStream and DataOutputStream.When the client logs out it sends its' name and message i.e. "User_Name has Logged out" and terminates the chatting.

ChatServer.java

```
import
java.net.*;
import
java.io.*;
public class
ChatServer
implements
Runnable {
private
ChatServer
Thread
clients[] =
new
ChatServer
Thread[50];
```

```
private
ServerSocke
t server =
null;
private
Thread
thread =
null;
private int
clientCount
= 0;
public
ChatServer(
int port) {
try
{
System.out.
println("bin
ding to
port" + port
+ ",please
wait ...");
server =
new
ServerSocke
t(port);
System.out.
println("ser
ver started:"
+ server);
thread =
new
Thread(this)
;
thread.start(
);
}
catch (IOException e) {
System.out.
println("can
not bind to
port" + port
+ ":" +
e.getMessag
e());
}
```

```
}
public void
run() {
while
(thread !=
null) {
try {
System.out.println("waiting for a client...");
addThread(server.accept());
}
catch (IOException e) {
System.out.println("server accept error:" + e);
if (thread !=
null) {
thread.stop(
);
thread =
null;
}
}
}
}
public void
stop() {
if (thread !=
null) {
thread.stop(
);
thread =
null;
}
}
private int
findClient(i
nt ID)
{
for (int i =
0;
i<clientCou
nt; i++)
if (clients[i].getID() == ID)
return i;
return -1;
}
public
synchronize
```

```
d void
handle(int
ID,String
input) {
if
(input.equal
s("quit")) {
clients[findClient(ID)].send("quit");
remove(ID);
} else
System.out.
println(ID +
":" + input);
for (int i =
0;
i<clientCou
nt; i++)
clients[i].se
nd(ID + ":"
+ input);
}
public
synchronize
d void
remove(int
ID) {
int pos =
findClient(I
D);
if (pos>= 0)
{
ChatServer
Thread
closing =
clients[pos];
System.out.
println("rem
oving client
thread:" +
ID + "at" +
pos);
if
(pos<client
Count - 1)
for (int i =
pos + 1;
```

```
i<clientCou
nt; i++)
clients[i - 1]
= clients[i];
clientCount-
-;
try {
closing.clos
e();
}
catch
(IOExceptio
n e) {
System.out.
println("erro
r closing
thread;" +
e);
}
closing.stop
();
}
}
private void
addThread(
Socket
socket) {
if
(clientCount
<clients.len
gth) {
System.out.
println("clie
nt
accepted:" +
socket);
clients[clien
tCount] =
new
ChatServer
Thread(this,
socket);
try {
clients[clien
tCount].ope
n();
```

```
clients[clien
tCount].start
();
clientCount
++;
} catch
(IOExceptio
n e) {
System.out.
println("erro
r opening
thread;" +
e);
}
} else
System.out.
println("clie
nt
refused;max
imum" +
clients.lengt
h +
"reached");
}
public static
void
main(String
a[]) {
ChatServer
server =
null;
if (a.length
!= 1)
System.out.
println("Usa
ge:javaChat
Server
Port");
else
server = new ChatServer(Integer.parseInt(a[0]));
}
}
```

ChatServerThread.java

```
import java.net.*;
import java.io.*;
public class ChatServerThread extends Thread {
private ChatServer server = null;
private Socket socket = null;
private DataInputStream In = null;
private int ID = -1;
private PrintStream Out = null;
public ChatServerThread(ChatServer serv,Socket sock) {
super();
server = serv;
socket = sock;
ID = socket.getPort();
}
public void send(String msg) {
Out.println(msg);
Out.flush();
}
public int getID() {
return ID;
}
public void run() {
System.out.println("Server thread" + ID + "running");        while (true) {
try {
server.handle(ID, In.readLine());
} catch (IOException e) {
System.out.println(ID + "error reading" + e.getMessage());
server.remove(ID);
stop();
}
}
}
public void open() throws IOException {
In = new DataInputStream(socket.getInputStream());
Out = new PrintStream(socket.getOutputStream());
}
public void close() throws IOException {
if (socket != null)
socket.close();
if (In != null)
In.close();
if (Out != null)
```

```
Out.close();
}
}
```

ChatClient .java

```
import java.net.*;
import java.io.*;

public class ChatClient implements Runnable {
   private ChatClientThread client = null;
   private Socket socket = null;
   private DataInputStream console = null;
   private Thread thread = null;
   private PrintStream Out = null;

   public ChatClient(String serverName, int serverPort) {
      System.out.println("Establishing connection, please wait...");
      try {
         socket = new Socket(serverName, serverPort);
         System.out.println("Connected: " + socket);
         console = new DataInputStream(System.in);
         Out = new PrintStream(socket.getOutputStream());

         if (thread == null) {
            client = new ChatClientThread(this, socket); // Separate thread for client
            thread = new Thread(this);
            thread.start();
         }
      } catch (UnknownHostException e) {
         System.out.println("Host unknown: " + e.getMessage());
      } catch (IOException ioe) {
         System.out.println("Unexpected exception: " + ioe.getMessage());
      }
   }

   @Override
   public void run() {
      while (thread != null) {
         try {
            Out.println(console.readLine()); // Send message to the server
```

```
            Out.flush();
        } catch (IOException e) {
            System.out.println("Sending error: " + e.getMessage());
            stop();
        }
    }
}

public void handle(String msg) {
    if (msg.equals("quit")) {
        System.out.println("Goodbye, press RETURN to exit...");
        stop();
    } else
        System.out.println(msg);  // Display message from server
}

public void stop() {
    if (thread != null) {
        thread.stop();  // Stop the thread (deprecated method; consider using better
thread handling)
        thread = null;
    }
    try {
        if (console != null) console.close();
        if (Out != null) Out.close();
        if (socket != null) socket.close();
    } catch (IOException e) {
        System.out.println("Error closing...");
    }
    if (client != null) {
        client.close();
        client.stop();
    }
}

public static void main(String args[]) {
    ChatClient client = null;
    if (args.length != 2) {
        System.out.println("Usage: java ChatClient <host> <port>");
    } else {
        client = new ChatClient(args[0], Integer.parseInt(args[1]));
    }
```

```
    }
}
```

ChatClientThread.java

```
import java.net.*;
import java.io.*;

public class ChatClientThread extends Thread {
    private Socket socket = null;
    private ChatClient client = null;
    private DataInputStream streamIn = null;

    public ChatClientThread(ChatClient client, Socket socket) {
        this.client = client;
        this.socket = socket;
        open();
        start();
    }

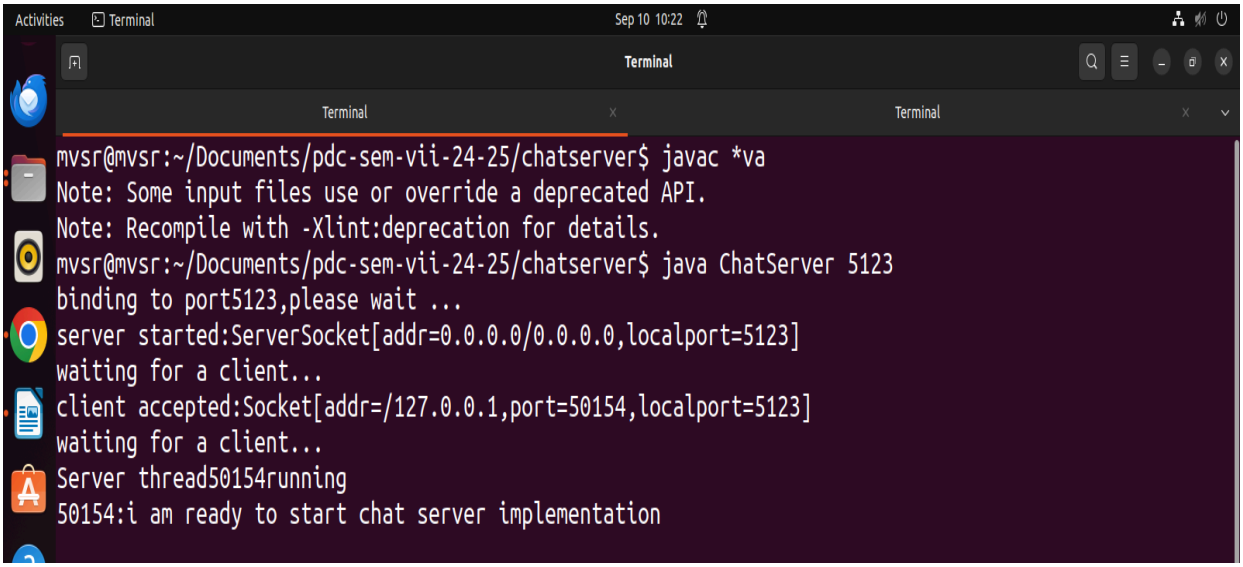
    public void open() {
        try {
            streamIn = new DataInputStream(socket.getInputStream());
        } catch (IOException e) {
            System.out.println("Error getting input stream: " + e);
            client.stop();
        }
    }

    public void close() {
        try {
            if (streamIn != null) streamIn.close();
        } catch (IOException e) {
            System.out.println("Error closing input stream: " + e);
        }
    }
```

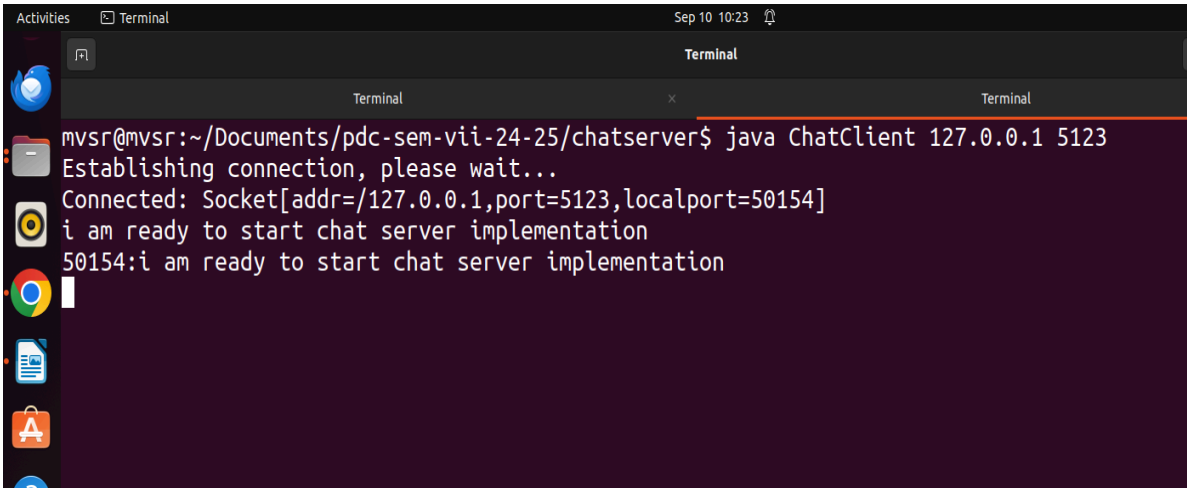
```
    @Override
    public void run() {
        while (true) {
            try {
                client.handle(streamIn.readLine());  // Read message from server
            } catch (IOException e) {
                System.out.println("Listening error: " + e.getMessage());
                client.stop();
            }
        }
    }
}
```

Sample Outputs:

ChatServer



ChatClient



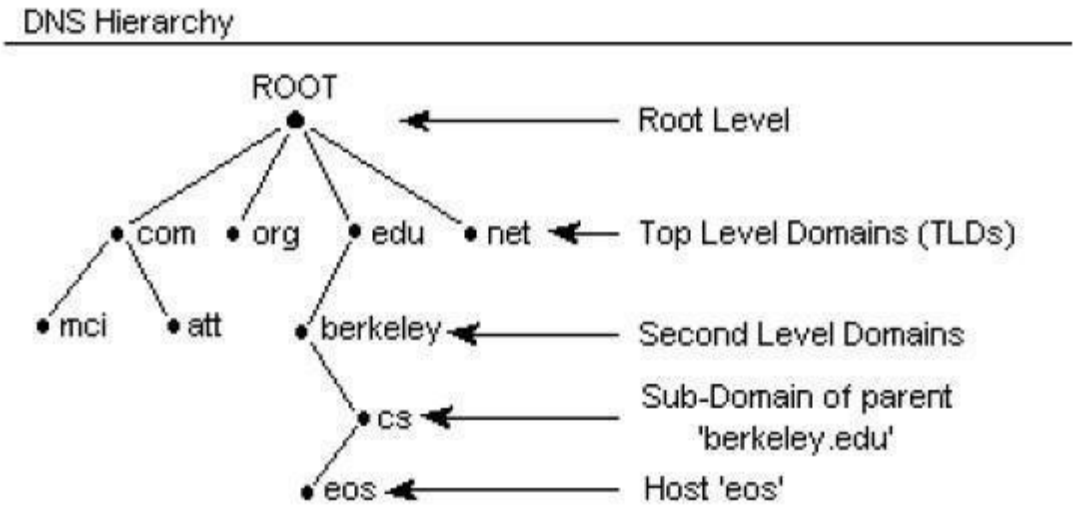
4. Domain Name Server Implementation Using Database

Introduction:

The **Domain Name System** (**DNS**) is a hierarchical naming system built on a distributed database for computers, services, or any resource connected to the Internet or a private network. The Domain Name System makes it possible to assign domain names to groups of Internet resources and users in a meaningful way, independent of each entity's physical location. Because of this, World Wide Web (WWW) hyperlinks and Internet contact information can remain consistent and constant even if the current Internet routing arrangements change.The Domain Name System distributes the responsibility of assigning domain names and mapping those names to IP addresses by designating iterative name servers for each domain.The Domain Name System is maintained by a distributed database system, which uses the clientserver model. The nodes of this database are the name servers. Each domain has at least one authoritative DNS server that publishes information about that domain and the name servers of any domains subordinate to it. The top of the hierarchy is served by the root name servers, the servers to query when looking up (*resolving*) a top level domain name.

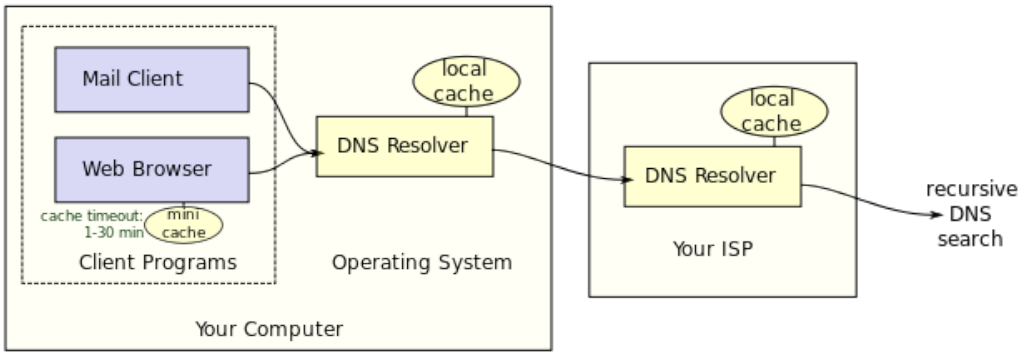
The Domain Name System also specifies the technical functionality of thedatabaseservice which is at its core. It defines the DNS protocol, a detailed specification of the data structures and data communication exchanges used in DNS, as part of theInternet Protocol Suite. Historically, other directory services preceding DNS were not scalable to large or global directories as they were originally based on text files, prominently theHOSTS.TXTresolver. DNS has been in wide use since the 1980s. The most common types of records stored in the DNS database are forDNS zoneauthority (SOA),IP addresses(A and AAAA),SMTPmail exchangers(MX),name servers(NS), pointers forreverse DNS lookups(PTR), anddomain name aliases(CNAME).

Domain name resolvers determine the domain name servers responsible for the domain name in question by a sequence of queries starting with the right-most (top-level) domain label.



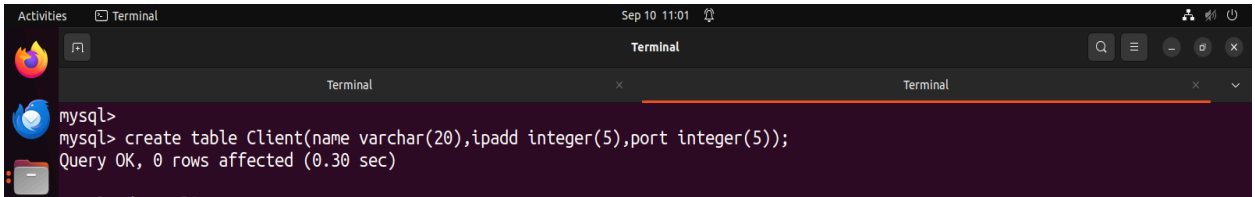
DNS resolvers

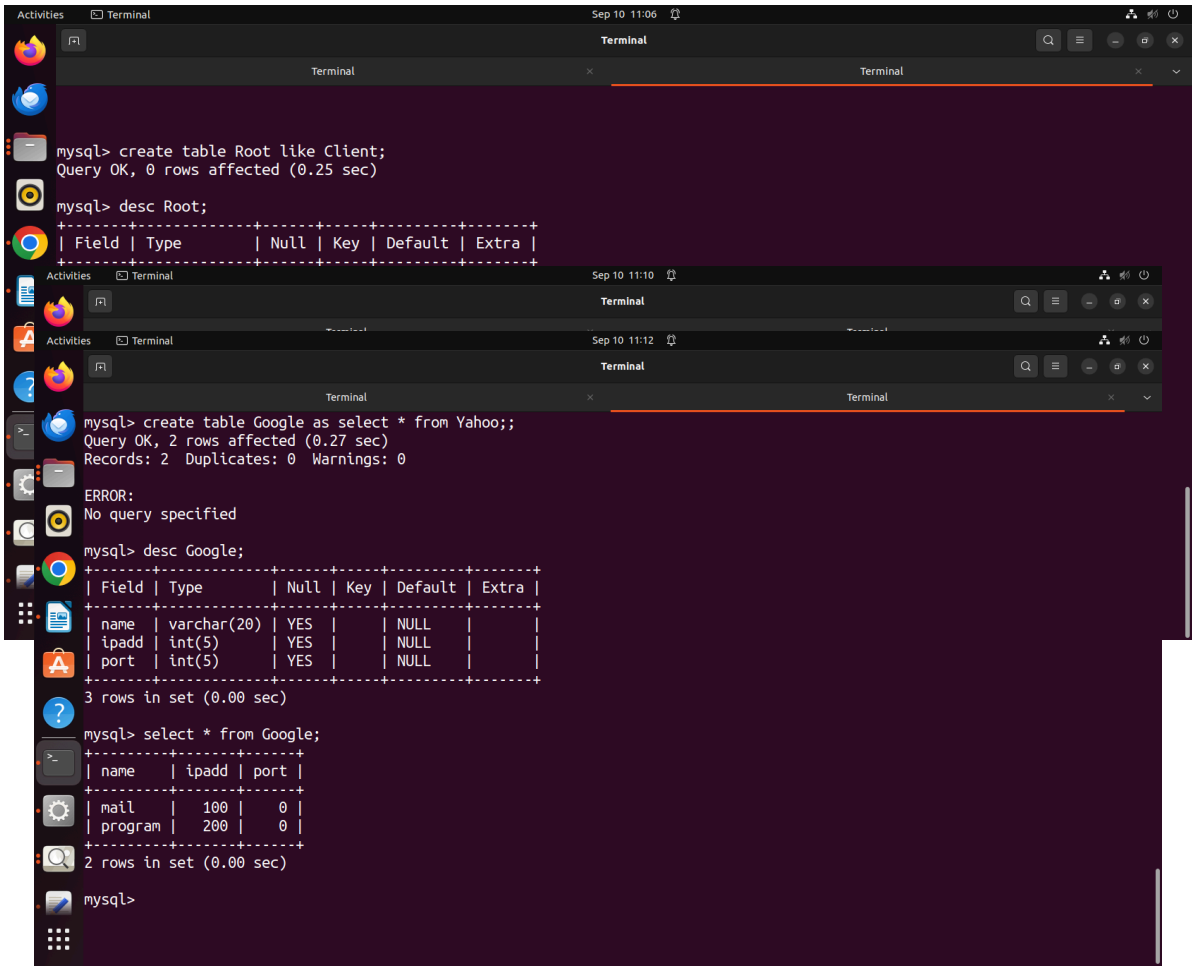
The client side of the DNS is called a DNS resolver. It is responsible for initiating and sequencing the queries that ultimately lead to a full resolution (translation) of the resource sought, e.g., translation of a domain name into an IP address.



Implementation:

- Design the table with in the database as specified in the next section.
- Implement individual server for each domain with in DNS.
- Each server program is now in waiting state.
- Implement client program which accepts URL from user, divides the URL into different domain and fetch the IPs of the domains from the corresponding servers.
- Combine the IPs received from different server and display them to the user.





Server1.java

```
import java.io.*;
import java.net.*;
import java.sql.*;
import
java.sql.Connectio
n;
import java.util.*;
class Server1 {
public static void
main(String a[]) {
ServerSocket
sock;
Socket client;
DataInputStream
input;
PrintStream ps;
String url, u, s;
Connection con;
Statement smt;
ResultSet rs;
try
{
s = u = "\0";
Class.forName("c
om.mysql.cj.jdbc.
```

```
Driver");
con =
DriverManager.ge
tConnection("jdbc
:mysql://localhost:
3307/dns", "root",
"");
smt =
con.createStateme
nt();
sock = new
ServerSocket(512
3);
while (true)
{
client =
sock.accept();
input = new
DataInputStream(
client.getInputStre
am());
ps = new
PrintStream(client
.getOutputStream(
));
url =
input.readLine();
System.out.println
("IN SERVER1
URL IS:" + url);
StringTokenizer st
= new
StringTokenizer(u
rl, ".");
while
(st.countTokens()
> 1)
s = s +
st.nextToken() +
".";
s = s.substring(0,
s.length()-
1).trim();
u =
st.nextToken();
rs =
smt.executeQuery
("select port,ipadd
from root where
name='" + u +
"'");
if (rs.next()) {
ps.println(Integer.
```

```
parseInt(rs.getStri
ng(1)));
ps.println(Integer.
parseInt(rs.getStri
ng(2)));
ps.println(s);
}
else {
ps.println("Illegal
address pleasr
check the spelling
again");
con.close();
}
}
}
catch(Exception
e){
System.err.println(
e);
}
}
}
```

Server2.java

```
import java.io.*;
import java.net.*;
import java.sql.*;
import
java.sql.Connectio
n;
import java.util.*;
class Server2
{
public static void
main(String a[])
{
ServerSocket
sock;
Socket client;
DataInputStream
input;
PrintStream ps;
String url, u, s;
Connection con;
Statement smt;
ResultSet rs;
try
{
s = u = "\0";
Class.forName("c
om.mysql.cj.jdbc.
```

```
Driver");
con =
DriverManager.ge
tConnection("jdbc
:mysql://localhost:
3307/dns", "root",
"");
smt =
con.createStateme
nt();
sock = new
ServerSocket(512
4);
while (true)
{
client =
sock.accept();
input = new
DataInputStream(
client.getInputStre
am());
ps = new
PrintStream(client
.getOutputStream(
));
url =
input.readLine();
System.out.println
("IN SERVER2
URL IS:" + url);
StringTokenizer st
= new
StringTokenizer(u
rl, ".");
while
(st.countTokens()
> 1)
s = s +
st.nextToken() +
".";
s = s.substring(0,
s.length() -
1).trim();
u =
st.nextToken();
rs =
smt.executeQuery
("select port,ipadd
from yahoo where
name='" + u +
"'");
if (rs.next())
{
```

```
ps.println(rs.getStr
ing(1));
ps.println(rs.getStr
ing(2));
ps.println(s);
}
else
{
ps.println("Illegal
address pleasr
check the spelling
again");
con.close();
}
}
}
catch (Exception
e)
{
System.err.println(
e);
}
}
}
```

Server3.java

```
import java.io.*;
import java.net.*;
import java.sql.*;
import java.sql.Connection;
import java.util.*;
 class Server3 {
public static void main(String a[]) {
ServerSocket sock;
Socket client;
DataInputStream input;
PrintStream ps;
String url, u, s;
Connection con;
Statement smt;
ResultSet rs;
  try {
s = u = "\0";
Class.forName("com.mysql.cj.jdbc.Driver");
con = DriverManager.getConnection("jdbc:mysql://localhost:3307/dns", "root", "");
  smt = con.createStatement();
```

```
sock = new ServerSocket(5125);
while (true) {
client = sock.accept();
input = new DataInputStream(client.getInputStream());
ps = new PrintStream(client.getOutputStream());
url = input.readLine();
System.out.println("IN SERVER3 URL IS:" + url);
StringTokenizer st = new StringTokenizer(url, ".");
while (st.countTokens() > 1)
s = s + st.nextToken() + ".";
s = s.substring(0, s.length() - 1).trim();
u = st.nextToken();
rs = smt.executeQuery("select port,ipadd from google where name='" + u + "'");
if (rs.next()) {
ps.println(rs.getString(1));
ps.println(rs.getString(2));
ps.println(s);
} else {
ps.println("Illegal address pleasr check the spelling again");
con.close();
}
}
} catch (Exception e) {
System.err.println(e);
}
}
}
```

Client.java

```
import java.io.*;
import java.net.*;
import java.sql.*;
import java.sql.Connection;
import java.util.*;
class Client {
public static void main(String a[]) {
Socket clisock;
DataInputStream input;
```

```
PrintStream ps;
String url, ip, s, u, p, str;
int pno = 5123;
Connection con;
Statement smt;
ResultSet rs;
boolean status = true;
try {
ip = s = p = u = "\0";
System.out.println("enter name to resolve");
BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
url = br.readLine();
Class.forName("com.mysql.cj.jdbc.Driver");
con = DriverManager.getConnection("jdbc:mysql://localhost:3307/dns", "root", "");
smt = con.createStatement();
while (status) {                s = "\0";
System.out.println("IN CLIENT URL IS:" + url);
StringTokenizer st = new StringTokenizer(url, ".");
if (st.countTokens() == 1) {
status = false;
}
while (st.countTokens() > 1)
s = s + st.nextToken() + ".";
s = s.substring(0, s.length() - 1).trim();
u = st.nextToken();
System.out.println("u=" + u);
rs = smt.executeQuery("select port,ipadd from client where name='" + u + "'");
if (rs.next()) {
p = rs.getString(1);
pno = Integer.parseInt(p);
str = rs.getString(2);
url = s;
ip = str + "." + ip;
} else {
System.out.println("pno=" + pno);
clisock = new Socket("127.0.0.1", pno);
input = new DataInputStream(clisock.getInputStream());
ps = new PrintStream(clisock.getOutputStream());
```

```
ps.println(url);
p = input.readLine();
pno = Integer.parseInt(p);
str = input.readLine();
url = input.readLine();
ip = str + "." + ip;
smt.executeUpdate("insert into client values('" + u + "','" + str + "','" + p + "')");
}
System.out.println("ip=" + ip);
}
ip = ip.substring(0, ip.length() - 1).trim();
System.out.println("ip address is:" + ip);
con.close();
} catch (Exception e) {
System.err.println(e);
}
}
}
```

Steps to Download xampp Manager for UBUNTU

1. Download xampp for ubuntu 64 bit
2. After installation go to terminal
   Cd Downloads
3.To give Executable Permission
chmod +x ./xa (press tab) – displays complete file name
4. sudo ./xa (press tab) and enter to install
give mvsr as a pswd

After installation

5.In one of the terminals, type the command **sudo /opt/lampp/manager***

→ Password :mvsr

6.Then the xamp server will be opened, click start all.
7. Go to https://locallhost.me/phpmyadmin in web browser and create 4 tables Client, Root,Yahoo,Google as shown in the “implementation” section above.
8.Put your database name as your roll number and make sure to replace **dns** (database name in the above programs) with your roll number.

Download mysqljdbc driver

Go to terminal set the path as rename the mysqljdbc driver name as type 4.jar

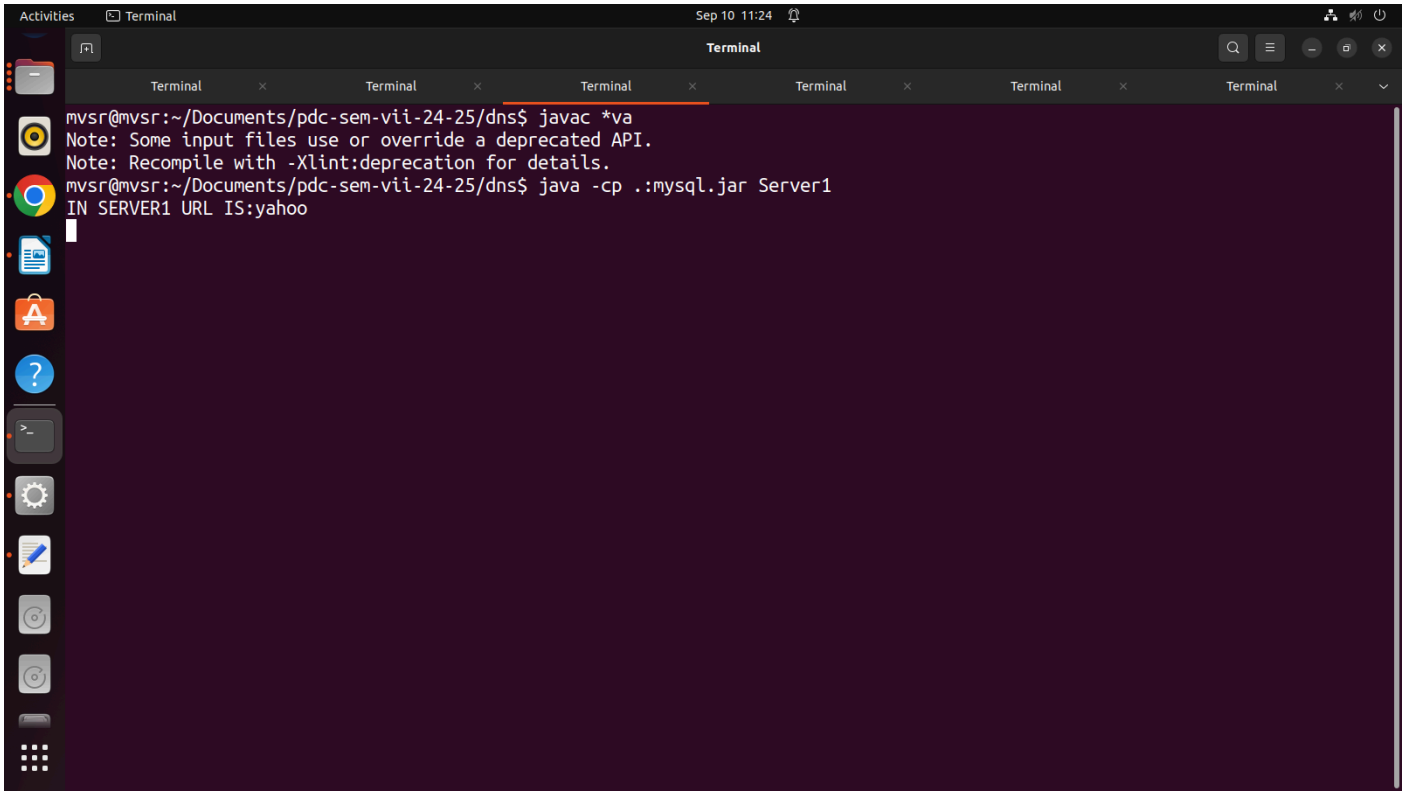
1.export export CLASSPATH=$CLASSPATH :type4.jar

javac Server1.java
java Server1

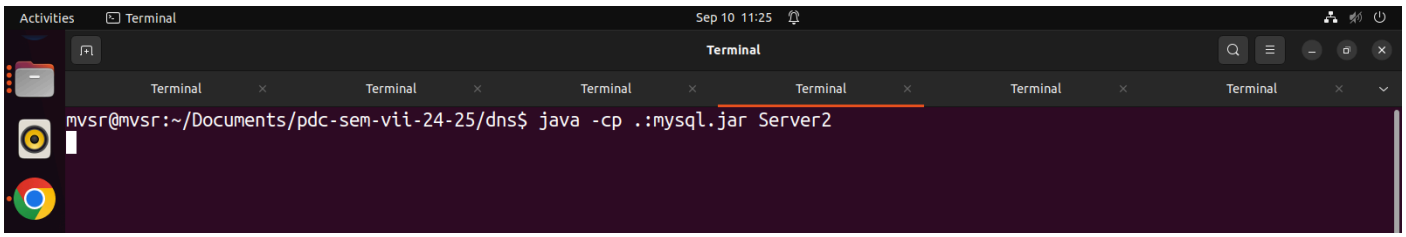
→ Enter name to resolve : google.com
→ 13.10

Sample Outputs:

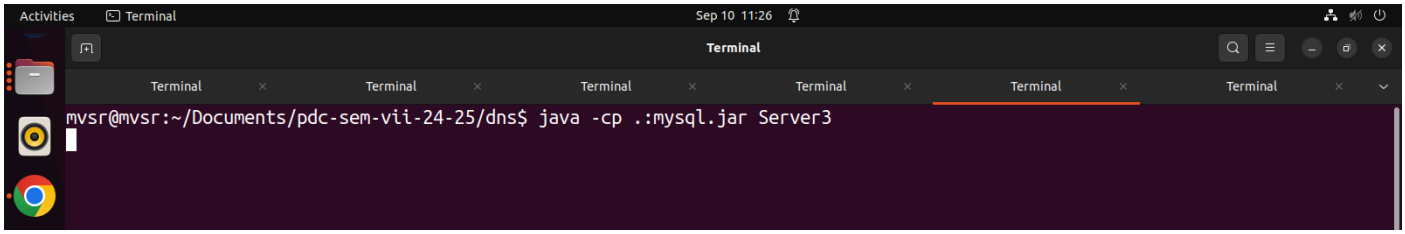
Server1



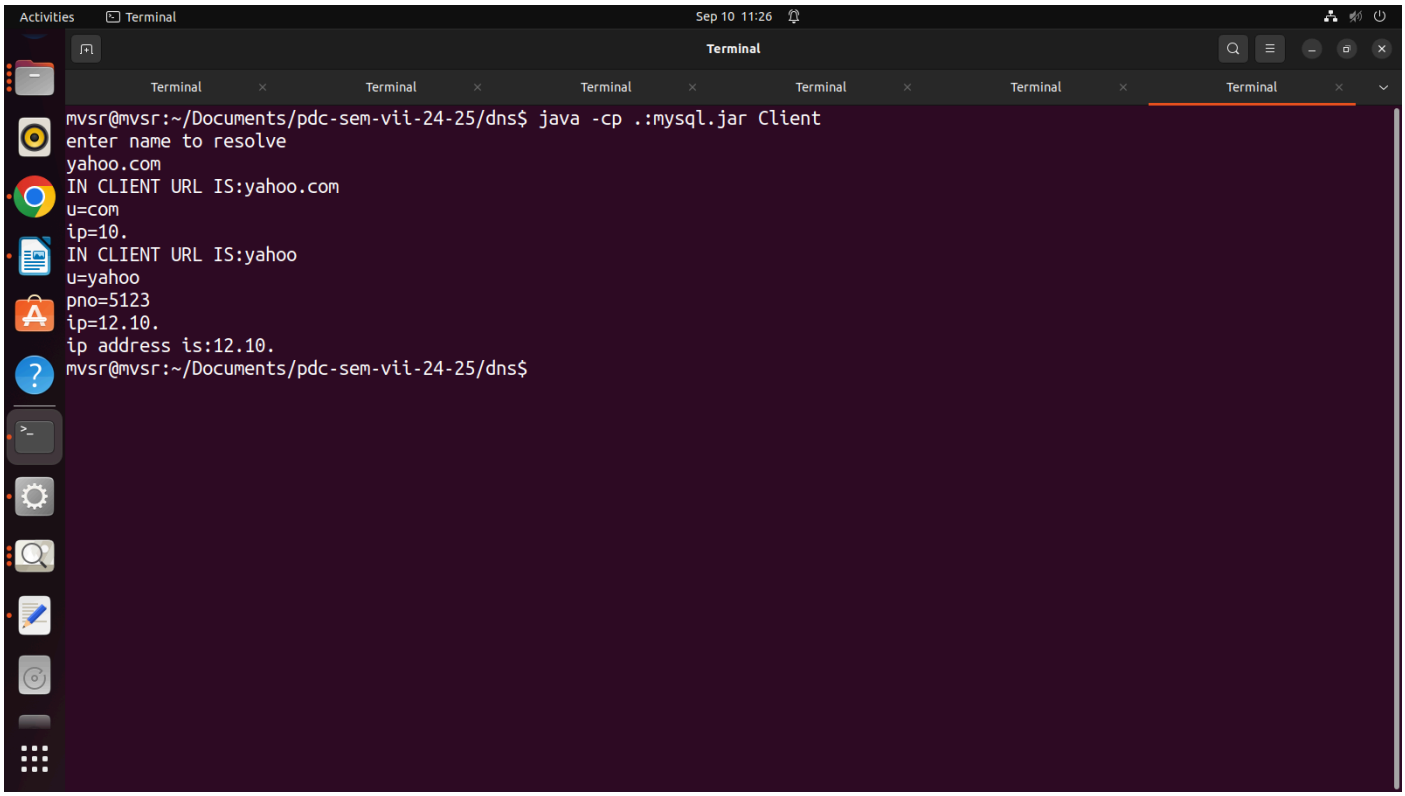
Server2



Server3



Client



5.Implementation of DNS using RMI

Introduction

This is a brief introduction to Java Remote Method Invocation (RMI). Java RMI is a mechanism that allows one to invoke a method on an object that exists in another address space. The other address space could be on the same machine or a different one. The RMI mechanism is basically an object-oriented RPC mechanism.

There are three processes that participate in supporting remote method invocation.

1. The *Client* is the process that is invoking a method on a remote object.

2. The *Server* is the process that owns the remote object. The remote object is an ordinary object in the address space of the server process.
3. The *Object Registry* is a name server that relates objects with names. Objects are *registered* with the Object Registry. Once an object has been registered, one can use the Object Registry to obtain access to a remote object using the name of the object.

RMI Stubs and Skeletons

- RMI uses stub and skeleton objects to provide the connection between the client and the remote object
- A stub is a proxy for a remote object which is responsible for forwarding method invocations from the client to the server where the actual remote object implementation resides
- A client's reference to a remote object, therefore, is actually a reference to a local stub. The client has a local copy of the stub object.
- A skeleton is a server-side object which contains a method that dispatches calls to the actual remote object implementation
- A remote object has an associated local skeleton object to dispatch remote calls to it.
- The skeleton object is automatically provided on the server side. A method can get a reference to a remote object
  - by looking up the remote object in some directory service. RMI provides a simple directory service called the RMI registry for this purpose
  - by receiving the remote object reference as a method argument or return value.

Steps To Develop an RMI Application

1. Design and implement the components of your distributed application
2. Define the remote interface(s)
3. Implement the remote object(s)
4. Implement the client(s)
5. Compile sources and generate stubs (and skeletons)
6. Make required classes network accessible.
7. Run the application.

DNSServer

```
import java.rmi.*;
import java.rmi.server.*;
import java.rmi.registry.*;
public class DNSServer extends UnicastRemoteObject implements DNSServerIntf
{
public DNSServer() throws RemoteException
{       super();
}
public String DNS(String s1) throws RemoteException
{         if(s1.equals("www.osmania.ac.in"))
```

```
        return "50.32.24.29";
          if(s1.equals("www.mvsrec.edu.in"))
                  return "90.82.44.89";
 if(s1.equals("www.jntu.ac.in"))
        return "150.32.64.20";
          if(s1.equals("www.yahoo.com"))
                  return "88.39.124.129";
else
        return "No Info about this address";
}
public static void main(String args[])throws Exception
{
                Registry r=LocateRegistry.createRegistry(8145);
                r.rebind("mvsrserver",new DNSServer());
                 System.out.println("server started...");
        }
}
```

DNSClient

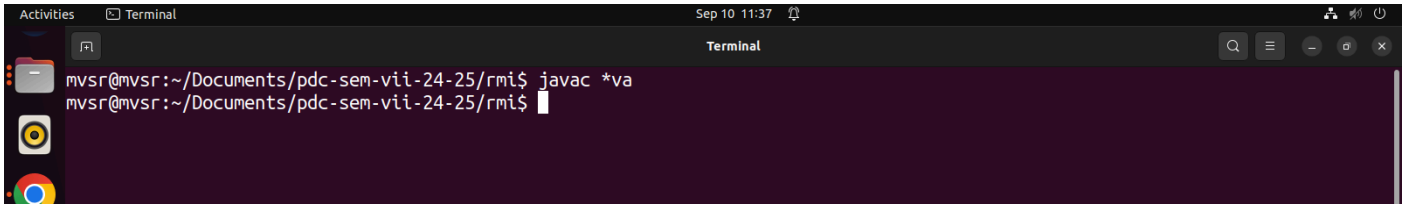
```
import java.rmi.*;
import java.rmi.registry.*;
public class DNSClient
{
public static void main(String args[])throws Exception
{
                Registry r=LocateRegistry.getRegistry("localhost",8145);
                DNSServerIntf d=(DNSServerIntf)r.lookup("mvsrserver");
                String str=args[0];
                System.out.println("The website name is:"+str);
                System.out.println("The IP Address is:"+d.DNS(str));
                        }
}
```

DNSServerIntf

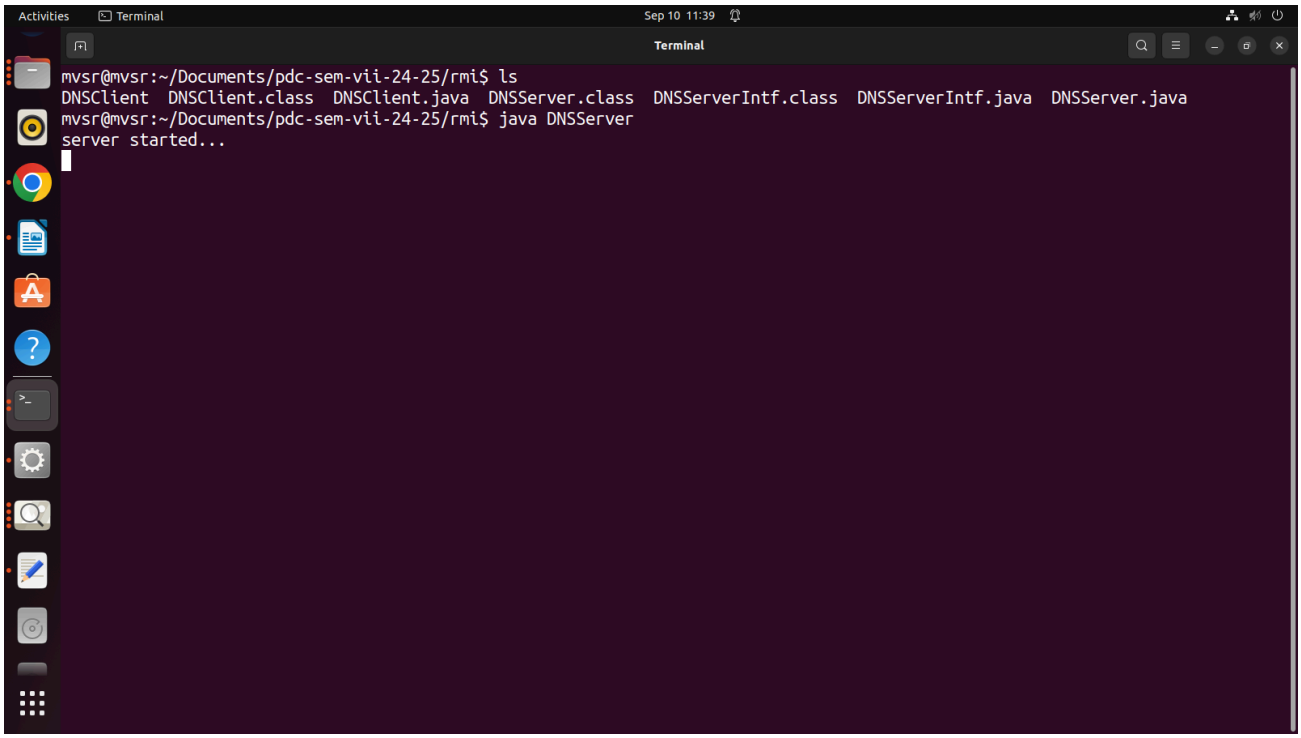
```
import java.rmi.*;
public interface DNSServerIntf extends Remote
{
public String DNS(String s1) throws RemoteException;
}
```

Sample Outputs:

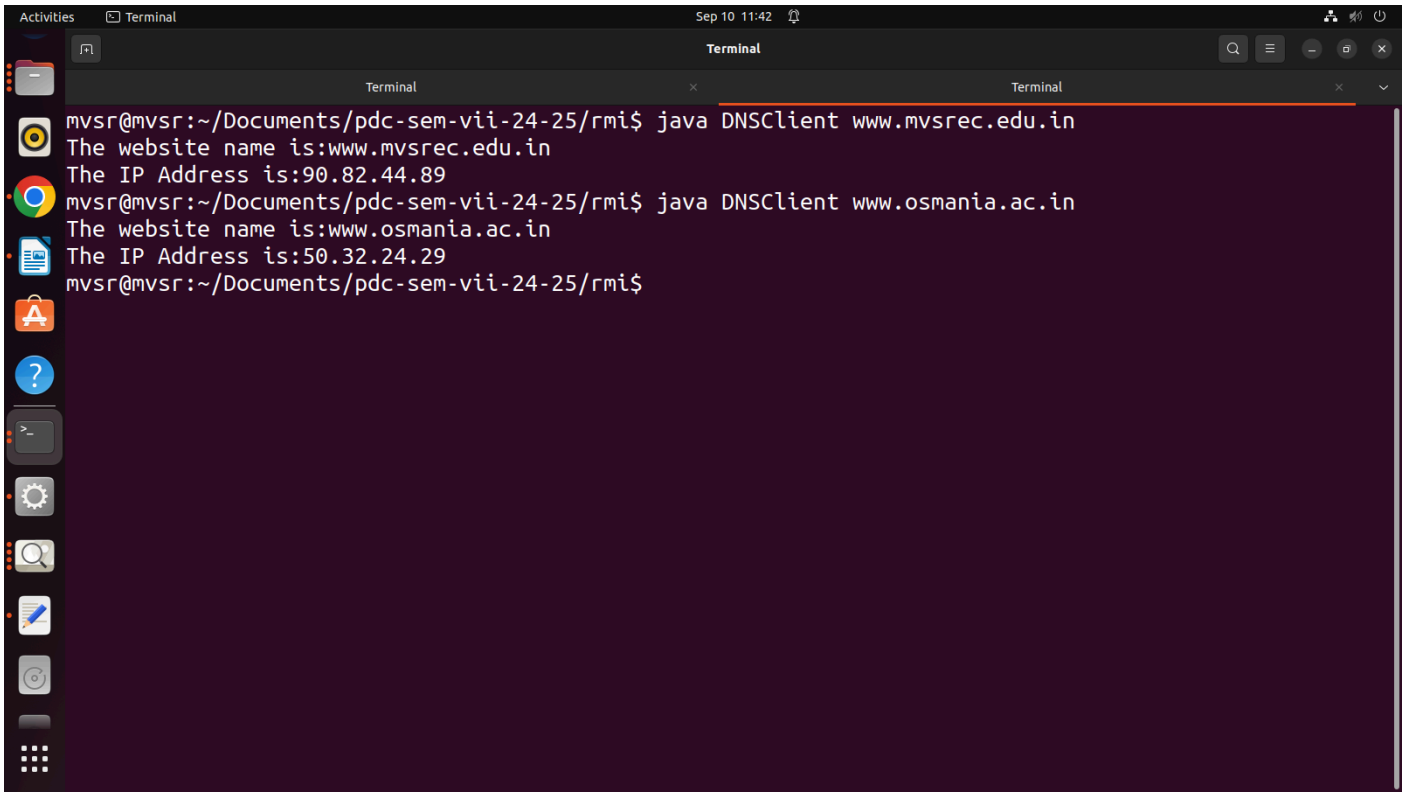
compilation:


Activities
Terminal
Sep 10 11:37
Terminal
mvsr@mvsr:~/Documents/pdc-sem-vii-24-25/rmi$ javac *va
mvsr@mvsr:~/Documents/pdc-sem-vii-24-25/rmi$

DNSServer


Activities
Terminal
Sep 10 11:39
Terminal
mvsr@mvsr:~/Documents/pdc-sem-vii-24-25/rmi$ ls
DNSClient DNSClient.class DNSClient.java DNSServer.class DNSServerIntf.class DNSServerIntf.java DNSServer.java
mvsr@mvsr:~/Documents/pdc-sem-vii-24-25/rmi$ java DNSServer
server started...

DNSClient



6.Implement a word count application which counts the number of occurrences of each word a large collection of documents Using Map Reduce model

Workflow of MapReduce consists of 5 steps:

1. Splitting – The splitting parameter can be anything, e.g. splitting by space, comma, semicolon, or even by a new line ('\n').
2. Mapping – as explained above.
3. Intermediate splitting – the entire process in parallel on different clusters. In order to group them in "Reduce Phase" the similar KEY data should be on the same cluster.
4. Reduce – it is nothing but mostly group by phase.
5. Combining – The last phase where all the data (individual result set from each cluster) is combined together to form a result.

Hadoop must be installed on your system with the Java SDK.

Steps

1. Open Eclipse> File > New > Java Project >(Name it –Samplewordcount) > Finish.
2. Right Click > New > Package (Name it - PackageDemo) > Finish.
3. Right Click on Package > New > Class (Name it - WordCount).
4. Add Following Reference Libraries:
    1. Right Click on Project > Build Path> Add External
        1. /usr/lib/hadoop-0.20/hadoop-core.jar

2. Usr/lib/hadoop-0.20/lib/Commons-cli-1.2.jar

The program consists of three classes:

- Driver class (Public, void, static, or main; this is the entry point).
- The Map class which extends the public class Mapper<KEYIN,VALUEIN,KEYOUT,VALUEOUT> and implements the Map function.
- The Reduce class which extends the public class Reducer<KEYIN,VALUEIN,KEYOUT,VALUEOUT> and implements the Reduce function.

6. Make a jar file

Right Click on Project> Export> Select export destination as Jar File > next> Finish.

7. Take a text file and move it into HDFS format:

To move this into Hadoop directly, open the terminal and enter the following commands:

[training@localhost~]$hadoop fs -put wordcountFilewordCountFile

8. Run the jar file:

(Hadoop jar jarfilename.jar packageName.ClassNamePathToInputTextFilePathToOutputDirectry)

[training@localhost~]$hadoop jar MRProgramsDemo.jar PackageDemo.WordCountwordCountFile MRDir1

9. Open the result:

[training@localhost~]$hadoop fs -ls MRDir1

Found 3 items

-rw-r--r-- 1 training supergroup 0 2016-02-23 03:36

/user/training/MRDir1/_SUCCESS drwxr-xr-x - training supergroup 0

2016-02-23 03:36 /user/training/MRDir1/_logs -rw-r--r-- 1 training supergroup

20 2016-02-23 03:36 /user/training/MRDir1/part-r-00000

[training@localhost~]$hadoop fs -cat MRDir1/part-r-00000

BUS 7

CAR 4

TRAIN 6

WordCount.java

```
package PackageDemo;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
```

```
import org.apache.hadoop.util.GenericOptionsParser;

public class WordCount
{
public static void main(String [] args) throws Exception
{
                Configuration c=new Configuration();
                String[] files=new GenericOptionsParser(c,args).getRemainingArgs();
                Path input=new Path(files[0]);
                Path output=new Path(files[1]);
                Job j=new Job(c,"wordcount");
                j.setJarByClass(WordCount.class);
                j.setMapperClass(MapForWordCount.class);
                j.setReducerClass(ReduceForWordCount.class);
                j.setOutputKeyClass(Text.class);
           j.setOutputValueClass(IntWritable.class);
FileInputFormat.addInputPath(j, input);
                FileOutputFormat.setOutputPath(j, output);
                System.exit(j.waitForCompletion(true)?0:1);
}

public static class MapForWordCount extends Mapper<LongWritable, Text, Text, IntWritable>
{
public void map(LongWritable key, Text value, Context con) throws IOException, InterruptedException  {
String line = value.toString();
String[] words=line.split(",");
for(String word: words )
{
Text outputKey = new Text(word.toUpperCase().trim());
IntWritableoutputValue = new IntWritable(1);
con.write(outputKey, outputValue);
}
}
}
public static class ReduceForWordCount extends Reducer<Text, IntWritable, Text, IntWritable>
{
public void reduce(Text word, Iterable<IntWritable> values, Context con) throws IOException,
InterruptedException
{ int sum = 0;
for(IntWritable value : values)
{
sum += value.get();
}
con.write(word, new IntWritable(sum));
}
}
}
```

PART-2 PARALLEL PROGRAMMING USING POSIX THREADS, OPENMP AND MPI

Posix: What and Why

The acronym POSIX stands for Portable Operating System Interface for uniX, and is used to refer to several standards organized and published by the Institute of Electrical and Electronics Engineers (IEEE) to provide a common basis for programmers to leverage operating system functionality. The API of interest here is the Posix threads API, often referred to as **pthreads** (it now occurs to me that the authors may have read Wodehouse, but most people have not, so the "p" is articulated: "P threads").

As you probably recall, threads usually differ from processes in that two threads in the same process share a single virtual address space. Aside from this difference, thread packages and the meaning of the term thread has varied widely. By the mid-90s, use of threads was becoming common, and several commercial implementations were avail- able (*e.g.*, in Sun's Solaris operating system). As more programmers started to use threads for server codes, portability between packages became more important. However, little or no consensus existed as to exactly how thread function- ality should be supported, how threads should be scheduled, which synchronization abstractions should be provided and how they should work, *etc.* To address this problem, a committee was created to define a common approach. In 1995, the committee produced the first Posix threads standard, IEEE POSIX 1003.1c, which standardized the abstrac tions and interfaces to a variety of synchronization and control mechanisms commonly used with parallelism based on shared memory threads.

As mentioned already, APIs such as Solaris threads had begun to see widespread use by the time the first standard appeared, and support for these models had been integrated into the corresponding operating systems. As a result, Posix implementations were for many years little more than glue sitting on top of the native threads for a given operating system. Even today, nearly a decade and a half later, many operating systems are not fully compliant with the standard. Linux, for example, does not support the idea of specifying the scheduling scope for a thread.

1.1 Incompatibilities with Threads

Although standards are often necessary for mature abstractions, threads posed a particularly pressing issue because integration of threads into most UNIX systems required changes to other common interfaces. The main problems in this regard were two-fold: undefined semantics for many standard C library routines when used by threads, and incompatibility between the C error model and threads. If individual operating systems vendors were required to solve these

problems on their own, the resulting solutions would be highly disparate and difficult to reconcile later.

Let's examine the library routine issues first. The behavior of a library routine is typically defined in terms of its interface: given certain inputs, the routine produces certain outputs, or indicates one of several error conditions. In a sequential program, such an input/output description suffices for using the routine. Introduction of threads, however, creates the possibility of several simultaneous (parallel) executions of any given routine. The meaning of routines not designed for this type of execution are not well-defined. Do they operate as though executed in series according to some (unknown) order? In general, no, but even that constraint, which can be achieved by simply wrapping the code implementing each routine with a routine-specific lock, is not always sufficient to make the routines useful.

A routine with semantics and an implementation that extend naturally to use in multiple threads is said to be **thread- safe**, or **reentrant**. The standard C library contains many routines that do not meet this criterion. One common problem is the use of pointers to static data as return values. For example, the `gethostbyname` routine for finding an Internet (IP) address for a given domain (*e.g.*, "www.illinois.edu") returns a pointer to a static block of data in the library. The caller passes in the domain name; the routine performs the translation, fills in a static data block, and returns a pointer to the block; and the caller then makes use of the data *before calling the routine again*. In a sequential system, this technique avoids the overhead associated with dynamic allocation by either the caller or callee. Delaying a subsequent call, of course, cannot be guaranteed easily in a threaded program. If we cast this exchange as a critical section, the critical section must include consumption of the result, which lies outside of the routine and can thus only be implemented by the caller. In other words, obtaining the expected behavior from these routines in a threaded program implies that the application itself must provide synchronization between its threads when using the routine.

Why not just change the interface? Millions of lines of codes used these routines by the time threads became an issue, thus changing their interfaces was not an acceptable solution. For most of these routines, the Posix standards created new versions that require that the caller pass a pointer to a private block of data, thus decoupling the execution of the routines by multiple threads. The names for these new routines are often simply the original routine name with the suffix "`r`" appended to indicate a reentrant version. In some cases, the standard introduced entirely new routines and names. Note that a caller can (and typically does) use automatic/stack allocation for the data block, then consumes the result before returning. Most Unix manual (`man`) pages now include a section on standards compliance and thread safety as well as notices to the effect that use of the older, non-reentrant routines is deprecated.

The UNIX error model also posed difficulties for threads. Most C library routines return -1 on failure (NULL for routines that return pointers) and set the `errno` variable to indicate which type of error occurred. But this variable is again in static storage in the C library, where another thread can overwrite it before the value stored by a failed invocation is consumed. To address this problem, most pthreads implementations use the C preprocessor to redirect accesses to the `errno` variable to thread-specific storage. The library implementation itself, of course, must also be re-compiled to make use of this trick, but most implementations allow either model, so code that pre-dates pthreads continues to compile and execute as before.

Posix standards also introduced a new error model to avoid perpetuating these types of problems in any new routines. All Posix routines return 0 on success, or a positive error code on failure. Return values such as pointers or integers are delivered as output arguments. In other words, the caller allocates space for the result and passes a pointer to the space. Although this type of interface adds a few instructions to most call sequences, processors were fast enough by the 1990s that most people agreed that the benefit to software engineering outweighed the cost to performance.

1.2 What to Standardize

The first Posix threads standard addressed the deficiencies in previous Unix interfaces, but also introduced new inter- faces for creating, managing, and controlling interactions between threads. For threads, the standard addressed how they are created, what they do, and what happens to them when they terminate. It also discussed methods by which one thread can terminate another asynchronously, an action called **cancellation**. A fairly standard mix of synchronization abstractions was also standardized, including mutexes, reader-writer locks, and condition variables. Semaphores had been standardized two years earlier as part of the Posix real-time standard, and the differences between the semaphore API and those of the other synchronization abstractions reflects the maturity of the standardization process. Interfaces for some other abstractions, such as barriers, were also specified by later versions of the pthreads standard, but are not required to be included in an implementation. Finally, the Posix threads standard specified how each of these abstractions interacted with the operating system scheduler, thread priorities, and other abstractions already present in most Unix implementations. For some interfaces, standardization took a little longer to get right: interactions between threads and signals, for example, were somewhat unstable for a couple of years before eventually reaching the fairly clear rules that exist in the latest version of the standard.

One unifying aspect that clearly differentiates the Posix threads standard from the earlier Posix standards such as the real-time standard is the creation of **attributes structures** that allow applications to create named classes of thread and synchronization variable behavior. As the Posix standard aimed to provide a generic thread interface, the abstractions covered—threads, mutexes, reader-writer locks, and condition variables—include controls to specify the behavior of instances. To avoid race conditions, most choices must be made when a given instance is created. Rather than pass each choice as an argument, the threads standard defines attributes structures that specify all options, and require that an appropriate attribute structure be passed to the routine that creates an instance.

Three benefits of the use of attributes structures are worth noting. First, passing a NULL pointer in place of an attributes structure provides the default behavior, which in many cases may be implementation-dependent. In contrast, all arguments passed to a function must be understood by anyone using the function, even if they need only know a constant value used to request default behavior. Second, as noted earlier, since attributes structures are simply variables like any other piece of data, applications can use the names of the attributes structures to refer to behavioral classes of instances of the corresponding abstraction. Obtaining the same benefit with per-choice arguments requires a set of variables for each behavioral class. Finally, future standards can extend the contents of the attributes structure and assign default options to the new choices *without changing the creation interface*, and thus without affecting existing pthreads code. This benefit cannot be obtained easily when arguments specifying the choices are passed separately.

The semaphore API does not use attributes, but instead uses an argument of the instance initialization function to specify a single choice for behavior. Although the benefit of updating the older interface to reflect the introduction of attributes seems obvious, enough code was probably written in the intervening two years using the initial semaphore API that any change would have undermined confidence in Posix standardization.

```
// thread management functions
  int pthread create (pthread t* new thread, const pthread
                      attr t* attr, void* (*thread main)(void*),
                      void* arg);
  pthread t pthread self ();
  int pthread join (pthread t thread, void**
  rval ptr);
  void pthread exit (void* ret val);

// cleanup functions
  void pthread cleanup push (void (*routine)(void* arg),
  void* arg); void pthread cleanup pop (int execute);

// cancellation functions
  int pthread cancel (pthread t
  thread); void pthread testcancel
  (void);
  int pthread setcanceltype (int type, int* old
  type); int pthread setcancelstate (int state,
  int* old state);
```

Figure 1: Basic Posix threads interfaces

Posix represents threads using an opaque type, `pthread t`. The routines for basic thread management appear in Figure 1. Each thread accepts one pointer as an argument, executes a single function, and returns a pointer as its result. Use of a pointer for arguments and return values allows communication of arbitrary amounts of information by simply creating a structure and using the address of the structure as the information passed to or from the thread in question. Use specifically of a void pointer avoids any warnings for implicit pointer conversions; C compilers silently convert any other pointer type to and from the void pointer type.

The function for thread creation is `pthread create`. Any thread can create (or **spawn**) a new thread, and a thread can obtain its own identifier using `pthread self`. The `attr` argument points to an attributes structure from which behavioral options for the thread are drawn. A NULL pointer sets all choices to default values. The function returns 0 when successful, in which case a thread identifier is stored in the location referenced by `new thread`, and the new thread starts executing the function `thread main` with the single argument `arg`.

Be careful not to make any assumptions about the relative execution order of the thread being created and the thread that calls the `pthread create` *function*. For example, if the new thread starts by walking over a data structure, be sure that the data structure is initialized before creating the thread. This type of mistake is particularly troublesome because the scheduling dynamics of your test environment may make bugs from the resulting race condition impossible to observe, while someone else's environment might make them impossible to avoid.

As shown by the signature above, the `thread main` function returns a void pointer. When a thread created with `pthread create` returns from its `thread main` function, the thread terminates, but the return value is retained by the operating system until another thread requests it. Asking for a thread's return value is called **joining** the thread, and is a blocking operation in Posix. Any thread can join with any other by calling `pthread join`. The calling thread does not return successfully from this function until the target thread has terminated. However, the call can fail due to deadlock prevention (cyclic join attempts) or targeting a non-existent or non-joinable thread. When successful, the target thread's return value is stored in the pointer referenced by `rval ptr`.

The pthreads standard also defines a block-structured exception handling mechanism. Cleanup

routines can be pushed onto a stack of such routines using `pthread cleanup push` and subsequently popped off using `pthread - cleanup pop`. When popping a cleanup routine, the `execute` argument specifies whether or not the cleanup routine should be called. If a thread terminates while the stack is non-empty, all routines on the stack are popped off and executed before termination.

A thread created with `pthread create` can also terminate by calling `pthread exit`, which handles the stack of cleanup routines and then returns with the specified value. Execution of a `return` statement in a thread function makes an implicit call to `pthread exit`. The natural way to implement this call is to simply start new threads with a stack frame for a routine that handles cleanups, and then to push a stack frame for the thread's main routine on top of the first stack frame. When a thread's main function returns, it effectively "calls" the cleanup execution routine.

The termination behavior discussed here does not extend to the initial program thread—the one that executes `main`. If the initial thread terminates in any manner, all remaining threads are terminated, and the program ends. Calling `exit`, `abort`, or similar functions from any thread also terminates the program.

2.1 Cancellation

A thread can also terminate in a third way: cancellation by another thread using `pthread cancel`. A joinable thread should be cancelled if at some point the work that it is performing becomes pointless. In a parallel search to find a single solution, for example, one might launch one thread per processor and then, when one thread finds a solution, cancel the others. Clearly one would not want to wait for all threads to find a solution, which is implied by joining. Cancellation creates race conditions and consistency issues because the thread making the cancellation call and the one being cancelled execute asynchronously. To avoid these problems, Posix allows threads that might be cancelled to select between an asynchronous model in which any critical sections must be protected by appropriate calls to prevent cancellation and a synchronous model in which cancellation is only possible at certain points in the code executed by the thread being cancelled. Cancellation is not instantaneous, and a canceled thread executes its stack of cleanup handlers before terminating. *Posix thus expects that another thread will still join with the cancelled thread to confirm that cancellation has completed.*

New threads start in the mode in which cancellation can only occur at points in the code specified by the thread. In other words, cancellation is possible, but is deferred until the thread itself indicates an appropriate time. To do so, the thread calls `pthread testcancel`.

If a thread can be cancelled at any point in its execution, it can indicate this fact to the operating system by calling `pthread setcanceltype` with type `PTHREAD CANCEL ASYNCHRONOUS` (the initial cancellation type is `PTHREAD CANCEL DEFERRED`); the call stores the previous value in the location referenced by `old type`.

Frequently, a thread that allows asynchronous cancellation must occasionally enter critical sections within which cancellation should be avoided. In this case, the thread can disable cancellation by calling `pthread setcancelstate` with state `PTHREAD CANCEL DISABLE` at the start of the critical section and `PTHREAD CANCEL ENABLE` at the end of the critical section. The call again returns the current value in the location referenced by `old state`, so a programmer can also create nested critical sections by simply restoring the previous state at the end of the critical section.

2.2 Thread Attributes

As mentioned earlier, Posix encapsulates other options for thread creation into an attributes structure—for a thread, a `pthread attr t`—that can be passed optionally when creating a thread. The routines for managing thread attributes appear in Figure 2. Choices available with attributes focus primarily on the scheduling policy for the new thread and on whether or not another thread will eventually join it. An attribute can be initialized to default options for all choices using `pthread attr init` and destroyed using `pthread attr destroy`.

One option encapsulated by the attributes structure specifies whether or not another thread will eventually join the thread. The problem faced by operating systems with threads is identical to that faced with processes: the return value must be retained in case another thread asks for it at some point in the future. The same is true for processes, and occasionally you can see "zombie" processes in an operating system, which have terminated, but remain visible until some other process asks how they terminated. With Posix threads, one can specify in advance that no other thread will ever request a new thread's return value, allowing the operating system to discard all information about the thread when it finishes executing its function.

```
// initialization and destruction
int pthread_attr_init (pthread_attr_t* attr);
int pthread_attr_destroy (pthread_attr_t*
attr);

// joinable/detached state; state can be
//      PTHREAD_CREATE_JOINABLE—thread must be joined eventually
//      PTHREAD_CREATE_DETACHED—thread cannot be joined/cancelled
int pthread_attr_getdetachstate (const pthread_attr_t* attr, int* state_ptr);
int pthread_attr_setdetachstate (pthread_attr_t* attr, int state);

// real-time scheduling inheritance; inher can be
//      PTHREAD_INHERIT_SCHED—inherit scheduling parameters from thread calling pthread_create
//      PTHREAD_EXPLICIT_SCHED—use scheduling parameters in attribute
int pthread_attr_getinheritsched (const pthread_attr_t* attr, int* inher_ptr);
int pthread_attr_setinheritsched (pthread_attr_t* attr, int inher);

// scheduling scope; scope can be
//      PTHREAD_SCOPE_SYSTEM—scheduled by OS scheduler
//      PTHREAD_SCOPE_PROCESS—scheduled at user-level (within process); see notes
int pthread_attr_getscope (const pthread_attr_t* attr, int* scope_ptr);
int pthread_attr_setscope (pthread_attr_t* attr, int scope);

// real-time scheduling policy; policy can be
//      SCHED_FIFO—first-in, first out (FIFO)
//      SCHED_RR—round-robin
//      SCHED_OTHER—other (typically OS-specific)
int pthread_attr_getschedpolicy (const pthread_attr_t* attr, int* policy_ptr);
int pthread_attr_setschedpolicy (pthread_attr_t* attr, int policy);

// real-time scheduling parameters (such as priority); see sched.h
int pthread_attr_getschedparam (const pthread_attr_t* attr,
                                struct sched_param* param_ptr);
int pthread_attr_setschedparam (pthread_attr_t* attr,
                                const struct sched_param* policy);
```

Figure 2: Interfaces for managing Posix thread attribute structures.

3.Synchronization Abstractions

The pthreads standard specifies several useful synchronization abstractions and routines to manage them. As men- tioned earlier, an attribute structure is also defined for each abstraction, and can be used to specify the behavior of each instance at creation time. A partial list of routines appears in Figure 3. This list includes only dynamic initializa- tion and destruction of attributes along with the routines used to specify process sharing of synchronization variables. Mutex attributes specify many other options, which are discussed later (and shown in Figure 4).

Synchronization variables in Posix can either be specific to a particular process (`PTHREAD PROCESS PRIVATE`) or shared by multiple processes (`PTHREAD PROCESS SHARED`). Variables that are private to a given process can still be used by all threads within that process, but cannot be used safely by threads in other processes.

```
// initialization and destruction
int pthread mutexattr init (pthread mutexattr t* m
attr); int pthread mutexattr destroy (pthread
mutexattr t* m attr); int pthread rwlockattr init
(pthread rwlockattr t* rw attr);
int pthread rwlockattr destroy (pthread rwlockattr t* rw attr);
int pthread condattr init (pthread condattr t* cv
attr); int pthread condattr destroy (pthread
condattr t* cv attr);

// inter- or intra-process sharing of variable; shared can be
//     PTHREAD PROCESS PRIVATE—use only by one process (multiple threads acceptable)
//     PTHREAD PROCESS SHARED—can be used by any process
int pthread mutexattr getpshared (const pthread mutexattr t* m
attr,
                                  int* shared ptr);
int pthread mutexattr setpshared (pthread mutexattr t* m attr,
int shared); int pthread rwlockattr getpshared (const pthread
rwlockattr t* rw attr,
                                   int* shared ptr);
int pthread rwlockattr setpshared (pthread rwlockattr t* rw attr,
int shared); int pthread condattr getpshared (const pthread
condattr t* cv attr,
                                 int* shared ptr);
int pthread condattr setpshared (pthread condattr t* cv attr, int
shared);
```

Figure 3: Interfaces for managing Posix synchronization variable attribute structures.

EXAMPLE PROGRAMS ON POSIX THREAD

1. PRINTING HELLO MESSAGE USING MULTIPLE PARALLEL THREADS

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

// Function that each thread will execute
void* thread_function(void* arg) {
    int thread_id = *((int*)arg);
    printf("Hello from thread %d!\n", thread_id);
    pthread_exit(NULL);
}

int main() {
    pthread_t threads[3];  // Array to hold thread identifiers
    int thread_ids[3];     // Array to hold thread IDs for passing to threads
    int num_threads = 3;

    // Create threads
    for (int i = 0; i < num_threads; i++) {
        thread_ids[i] = i;  // Assign unique ID to each thread
        int rc = pthread_create(&threads[i], NULL, thread_function, (void*)&thread_ids[i]);
        if (rc) {
            fprintf(stderr, "Error creating thread %d: %d\n", i, rc);
            exit(EXIT_FAILURE);
        }
    }

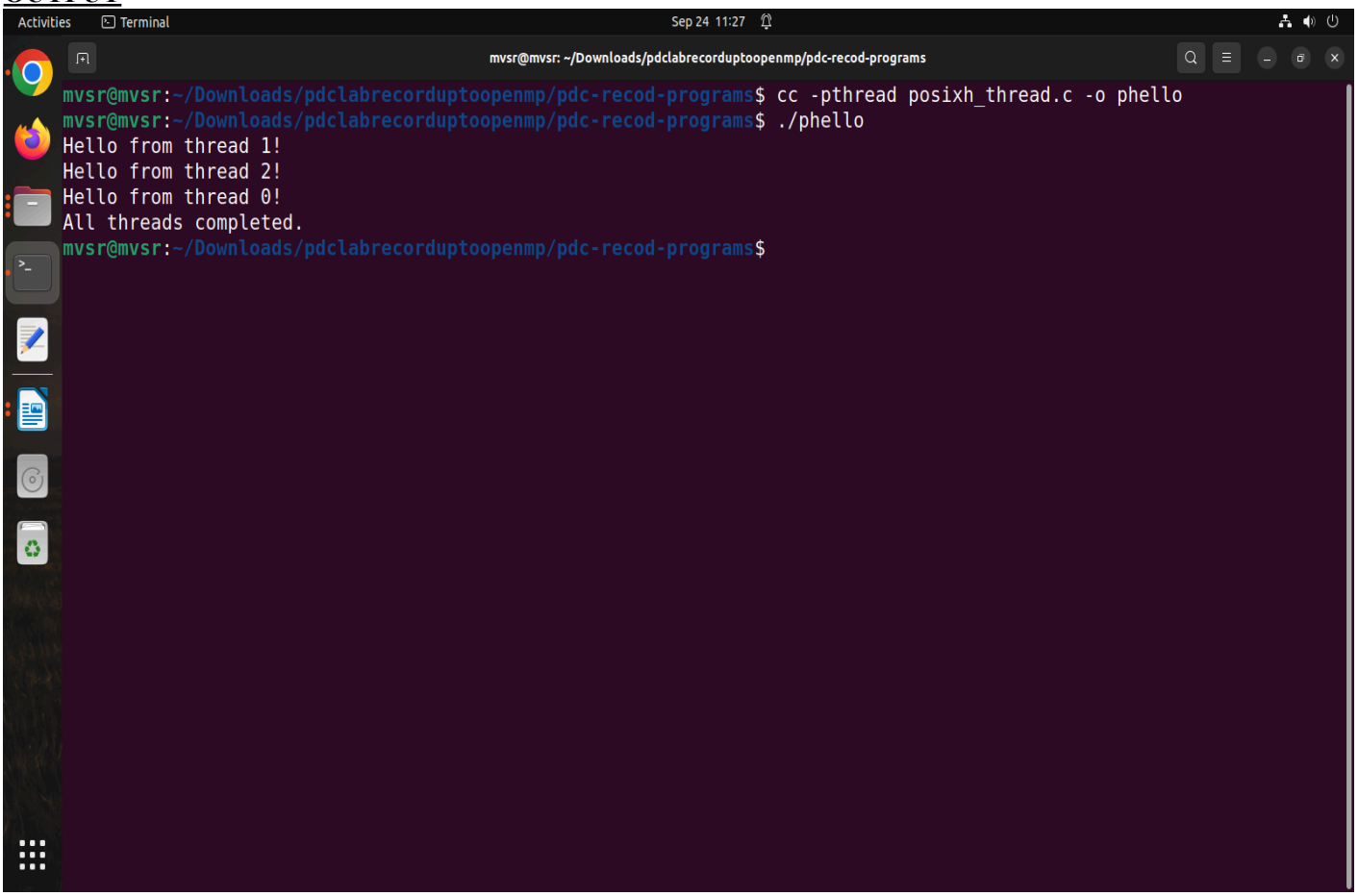
    // Wait for all threads to complete
    for (int i = 0; i < num_threads; i++) {
        int rc = pthread_join(threads[i], NULL);
        if (rc) {
            fprintf(stderr, "Error joining thread %d: %d\n", i, rc);
            exit(EXIT_FAILURE);
        }
    }

    printf("All threads completed.\n");
    return 0;
}

//Compilation: To compile this program, use the gcc compiler and link with the pthread library:
//cc -pthread posixh_thread.c -o phello


//   Execution:Run the compiled program:
//  ./phello
```

OUTPUT



2. PROGRAM FOR MERGE SORT USING POSIX THREAD

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define MAX 100000  // Maximum number of elements

int arr[MAX];
int temp[MAX];
int n;

typedef struct {
```

```
    int left;
    int right;
} ThreadData;

void merge(int left, int mid, int right) {
    int i = left, j = mid + 1, k = left;

    while (i <= mid && j <= right) {
        if (arr[i] <= arr[j]) {
            temp[k++] = arr[i++];
        } else {
            temp[k++] = arr[j++];
        }
    }

    while (i <= mid) {
        temp[k++] = arr[i++];
    }

    while (j <= right) {
        temp[k++] = arr[j++];
    }

    for (i = left; i <= right; i++) {
        arr[i] = temp[i];
    }
}

void *merge_sort(void *arg) {
    ThreadData *data = (ThreadData *)arg;
    int left = data->left;
    int right = data->right;

    if (left < right) {
        int mid = left + (right - left) / 2;

        ThreadData left_data = {left, mid};
        ThreadData right_data = {mid + 1, right};

        pthread_t left_thread, right_thread;

        pthread_create(&left_thread, NULL, merge_sort, &left_data);
        pthread_create(&right_thread, NULL, merge_sort, &right_data);

        pthread_join(left_thread, NULL);
        pthread_join(right_thread, NULL);

        merge(left, mid, right);
    }

    return NULL;
}
```

```
int main() {
    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter the elements:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    ThreadData data = {0, n - 1};
    pthread_t main_thread;

    pthread_create(&main_thread, NULL, merge_sort, &data);
    pthread_join(main_thread, NULL);

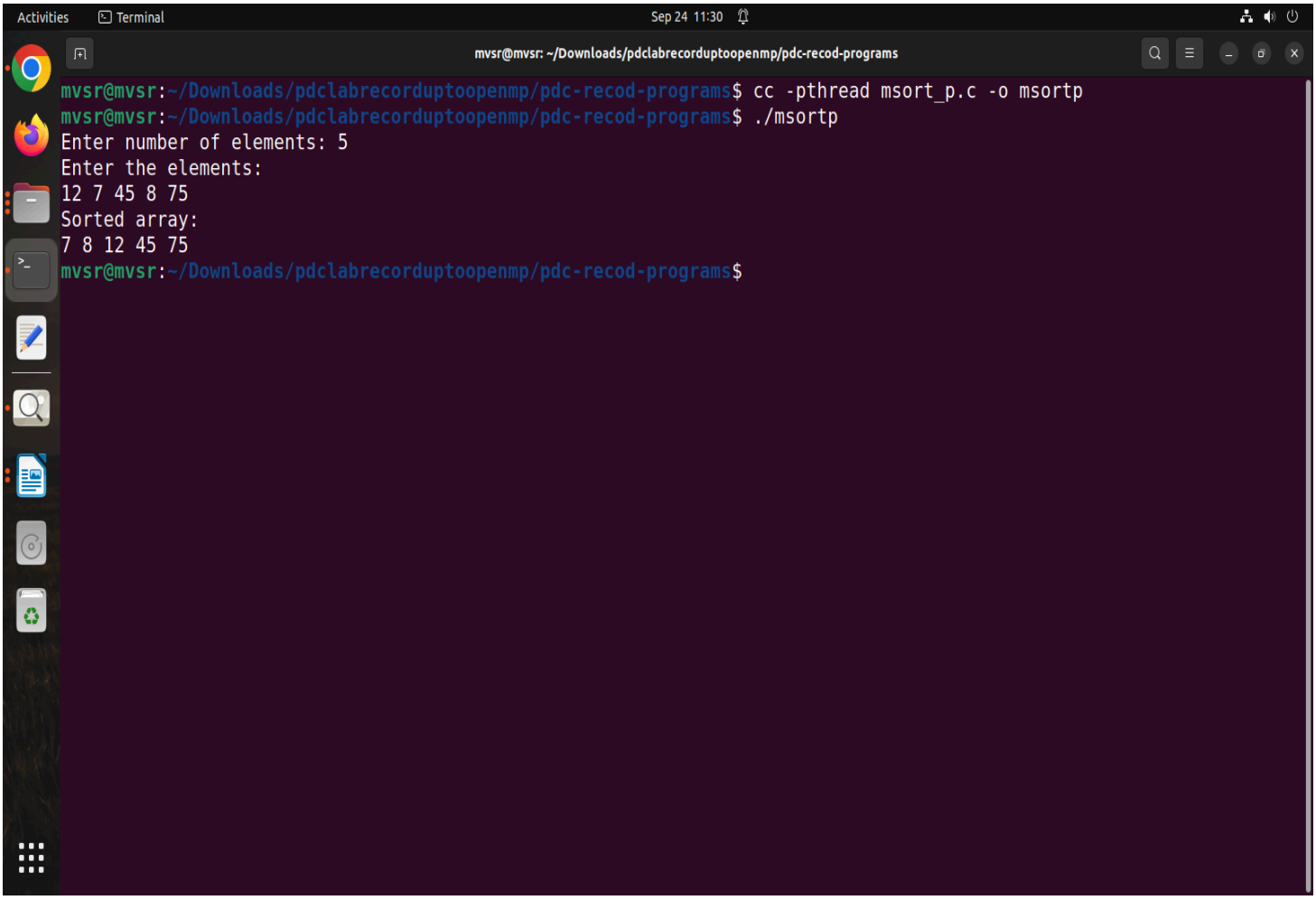
    printf("Sorted array:\n");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    return 0;
}

//  Compilation: To compile this program, use the cc compiler and link with the pthread library:
//  cc -pthread msort_p.c -o msortp


//  Execution:Run the compiled program:
//   ./msortp
```

Output:



OpenMP AND MPI(PART-2)

INTRODUCTION

Parallel processing has been in common use for decades. To deal with all types of grand challenges we must need High Performance Computing / Parallel Processing. Vary broadly; Parallel Processing can be achieved by two approaches, Hardware Approach and Software Approach and this Lab manual of Parallel Processing has been designed accordingly.

In Software approach, a parallel resource administration and management software is configured on systems / computers connected through LAN. All the hardware and software resources are combined logically to present a single system image to the programmer and user. Clusters and Grids are the examples of this approach. This environment is also refereed to as Distributed Memory environment. All

nodes / processors have their own local memories. The processors can communicate and share the resources using Message Passing. This approach is cheap and scalable but difficult to program, manage and secure.

In Hardware approach, all the processors or execution units are placed on the same motherboard sharing a common memory and other resources on the board. This approach is referred to as Shared Memory Architecture. This approach is expensive but much faster and easy to program. SMPs and Multi Core processors are examples of this system.

Programming a parallel system is not as easy as programming single processor systems. There are many considerations like details of the underlying parallel system, processors interconnection, use of the correct parallel programming model and selection of parallel language which makes the parallel programming more difficult. This lab manual is focused on writing parallel algorithms and their programming on distributed and shared memory environments.

Part (A) of PC LAB deal with SMP and Multi-Core systems programming. laboratory sessions are based on the theory and implementation of OpenMP directives and their clauses. Environment variables related to the OpenMP API are also discussed in the end.

Part (B) of PC LAB is based on Cluster Programming. A four node Linux based cluster using MPICH is used for programming. Lab starts with the basics of MPI and MPICH. The next two labs proceed with the communication among the parallel MPI processes. Third, fourth and fifth labs deal with the MPI collective operations. In the final lab some non blocking parallel operations are explored.

PART A: ***Basics of OpenMP API (Open Multi-Processor API)***

THEORY

OpenMP

OpenMP is a portable and standard Application Program Interface (API) that may be used to explicitly direct multi-threaded, shared memory parallelism

OpenMP attempts to standardize existing practices from several different vendor-specific shared memory environments. OpenMP provides a portable shared memory API across different platforms including DEC, HP, IBM, Intel, Silicon Graphics/Cray, and Sun. The languages supported by OpenMP are FORTRAN, C and C++. Its main emphasis is on performance and scalability.

Goals of OpenMP

- **Standardization:** Provide a standard among a variety of shared memory architectures/platforms
- **Lean and Mean:** Establish a simple and limited set of directives for programming shared memory machines. Significant parallelism can be implemented by using just 3 or 4

directives.

- **Ease of Use:** Provide capability to incrementally parallelize a serial program, unlike message-passing libraries which typically require an all or nothing approach and also provide the capability to implement both coarse-grain and fine-grain parallelism
- **Portability:** Supports Fortran (77, 90, and 95), C, and C++

OpenMP Programming Model

Shared Memory, Thread Based Parallelism:

- OpenMP is based upon the existence of multiple threads in the shared memory programming paradigm. A shared memory process consists of multiple threads.

Explicit Parallelism:

- OpenMP is an explicit (not automatic) programming model, offering the programmer full control over parallelization.

Fork - Join Model:

- OpenMP uses the fork-join model of parallel execution:

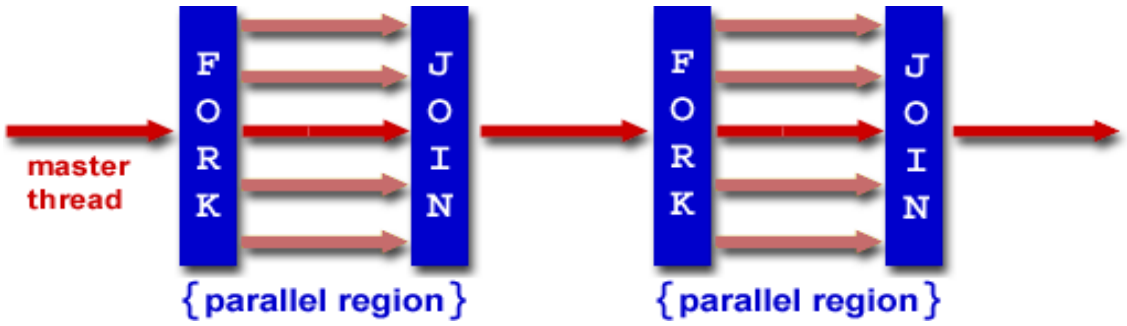


Figure 8.1 Fork and Join Model

- All OpenMP programs begin as a single process: the **master thread**. The master thread executes sequentially until the first **parallel region** construct is encountered.
- **FORK:** the master thread then creates a ***team*** of parallel threads
- The statements in the program that are enclosed by the parallel region construct are then executed in parallel among the various team threads
- **JOIN:** When the team threads complete the statements in the parallel region construct, they synchronize and terminate, leaving only the master thread

Compiler Directive Based:

- Most OpenMP parallelism is specified through the use of compiler directives which are imbedded in C/C++ or Fortran source code.

Nested Parallelism Support:

- The API provides for the placement of parallel constructs inside of other parallel constructs.
- Implementations may or may not support this feature.

Dynamic Threads:

- The API provides for dynamically altering the number of threads which may used to execute different parallel regions.
- Implementations may or may not support this feature.

I/O:

- OpenMP specifies nothing about parallel I/O. This is particularly important if multiple threads attempt to write/read from the same file.
- If every thread conducts I/O to a different file, the issues are not as significant.
- It is entirely up to the programmer to insure that I/O is conducted correctly within the context of a multi-threaded program.

Components of OpenMP API

- Comprised of three primary API components:
  - Compiler Directives
  - Runtime Library Routines
  - Environment Variables

C / C++ - General Code Structure

```
#include <omp.h>
main ()
{

      int var1, var2, var3;

      Serial code


            #pragma omp parallel private(var1, var2) shared(var3)

            {
                        Parallel section executed by all threads
                              .
                              .
                              .
                        All threads join master thread and disband
              }

       Resume serial code
}
```

KARACHI
ENGINEERING DEPARTMENT

Important terms for an OpenMP environment

Construct: A construct is a statement. It consists of a directive and the subsequent structured block. Note that some directives are not part of a construct.

directive: A C or C++ **#pragma** followed by the **omp** identifier, other text, and a new line. The directive specifies program behavior.

Region: A dynamic extent.

dynamic extent: All statements in the *lexical extent*, plus any statement inside a function that is executed as a result of the execution of statements within the lexical extent. A dynamic extent is also referred to as a *region*.

lexical extent: Statements lexically contained within a *structured block*.

structured block: A structured block is a statement (single or compound) that has a single entry and a single exit. No statement is a structured block if there is a jump into or out of that statement. A compound statement is a structured block if its execution always begins at the opening **{** and always ends at the closing **}**. An expression statement, selection statement, iteration statement is a structured block if the corresponding compound statement obtained by enclosing it in **{** and **}**would be a structured block. A jump statement, labeled statement, or declaration statement is not a structured block.

Thread: An execution entity having a serial flow of control, a set of private variables, and access to shared variables.

master thread: The thread that creates a team when a *parallel region* is entered.

serial region: Statements executed only by the *master thread* outside of the dynamic extent of any *parallel region*.

parallel region: Statements that bind to an OpenMP parallel construct and may be executed by multiple threads.

Variable: An identifier, optionally qualified by namespace names, that names an object.

Private: A private variable names a block of storage that is unique to the thread making the

reference. Note that there are several ways to specify that a variable is private: a definition within a parallel region, a threadprivate directive, a private, firstprivate, lastprivate, or reduction clause, or use of the variable as a forloop control variable in a for loop immediately following a for or parallel for directive.

Shared: A shared variable names a single block of storage. All threads in a team that access this variable will access this single block of storage.

Team: One or more threads cooperating in the execution of a construct.

Serialize: *To execute a parallel construct with a team of threads consisting of only a single thread (which is the master thread for that parallel construct), with serial order of execution for the statements within the structured block (the same order as if the block were not part of a parallel construct), and with no effect on the value returned by omp_in_parallel() (apart from the effects of any nested parallel constructs).*

Basic OpenMp example programs

1.Vector addition

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

#define N 4 // Size of the matrix and vector

int main() {
   // Allocate memory for the matrix and vectors
   int A[N][N], x[N], y[N];

   // Initialize matrix A and vector x
   for (int i = 0; i < N; i++) {
      x[i] = i + 1; // Vector initialization (1, 2, 3, ...)
      for (int j = 0; j < N; j++) {
         A[i][j] = (i + 1) * (j + 1); // Matrix initialization
      }
   }

   // Print matrix A
   printf("Matrix A:\n");
   for (int i = 0; i < N; i++) {
      for (int j = 0; j < N; j++) {
         printf("%d ", A[i][j]);
      }
      printf("\n");
   }

   // Print vector x
   printf("Vector x:\n");
```

```
    for (int i = 0; i < N; i++) {
        printf("%d ", x[i]);
    }
    printf("\n");

    // Start timing
    double start_time = omp_get_wtime();

    // Perform matrix-vector addition in parallel
    #pragma omp parallel for
    for (int i = 0; i < N; i++) {
        y[i] = 0; // Initialize the result vector
        for (int j = 0; j < N; j++) {
            y[i] += A[i][j]; // Sum the elements of the row
        }
        y[i] += x[i]; // Add corresponding element of vector x
    }

    // End timing
    double end_time = omp_get_wtime();
    printf("Time taken for matrix-vector addition: %f seconds\n", end_time - start_time);

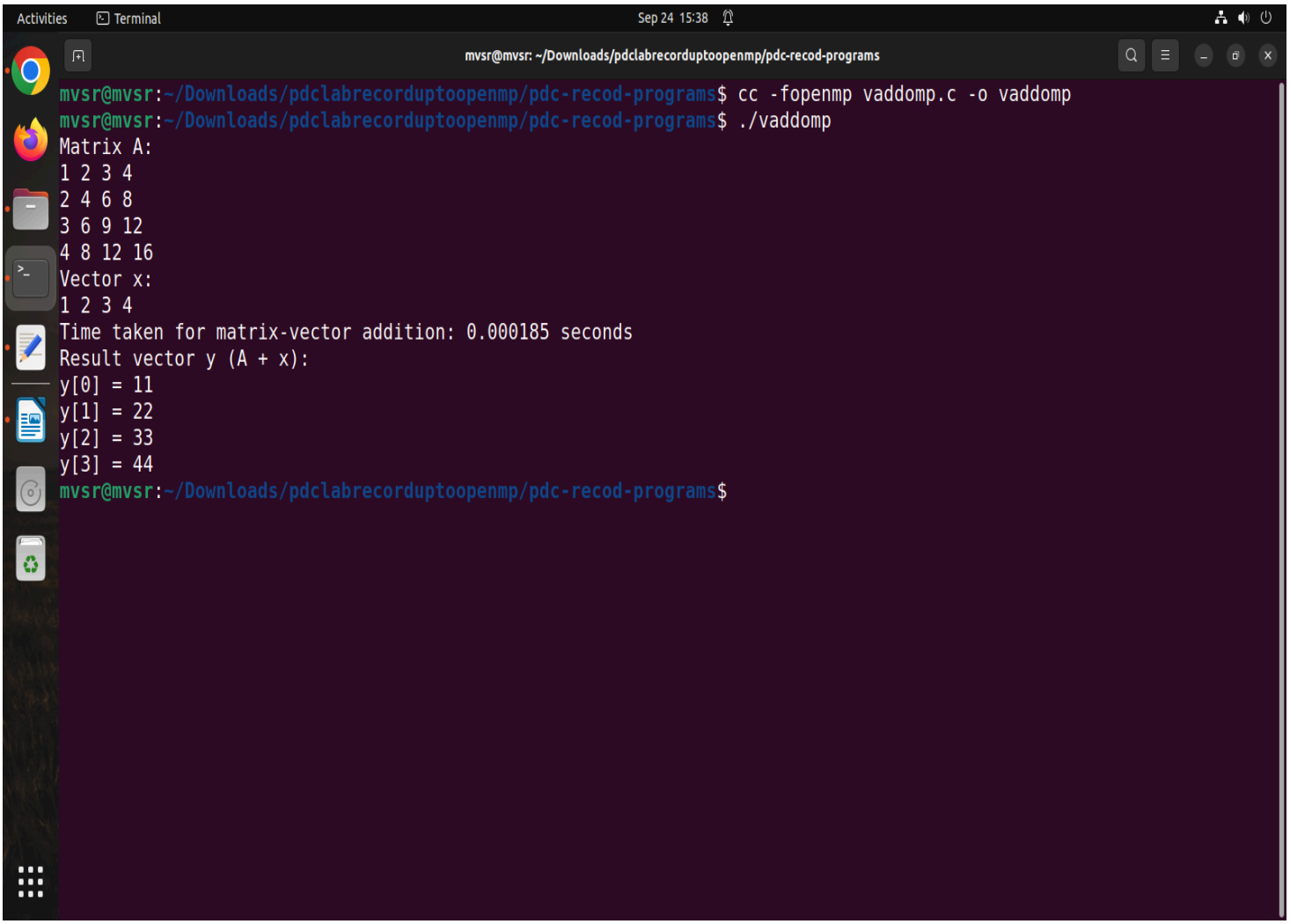
    // Print the result vector
    printf("Result vector y (A + x):\n");
    for (int i = 0; i < N; i++) {
        printf("y[%d] = %d\n", i, y[i]);
    }

    return 0;
}


// Compilation: To compile this program with OpenMP, use the following command:
// cc -fopenmp vaddomp.c -o vaddomp

//EXECUTION
// ./vaddomp
```

OUTPUT:


Activities
Terminal
Sep 24 15:38
mvsr@mvsr: ~/Downloads/pdclabrecorduptoopenmp/pdc-recod-programs
mvsr@mvsr:~/Downloads/pdclabrecorduptoopenmp/pdc-recod-programs$ cc -fopenmp vaddomp.c -o vaddomp
mvsr@mvsr:~/Downloads/pdclabrecorduptoopenmp/pdc-recod-programs$ ./vaddomp
Matrix A:
1 2 3 4
2 4 6 8
3 6 9 12
4 8 12 16
Vector x:
1 2 3 4
Time taken for matrix-vector addition: 0.000185 seconds
Result vector y (A + x):
y[0] = 11
y[1] = 22
y[2] = 33
y[3] = 44
mvsr@mvsr:~/Downloads/pdclabrecorduptoopenmp/pdc-recod-programs$

2. Vector multiplication

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

#define N 4 // Size of the matrix and vector

int main() {
   // Allocate memory for the matrix and vector
   int A[N][N], x[N], y[N];

   // Initialize matrix A and vector x
   for (int i = 0; i < N; i++) {
      x[i] = i + 1; // Vector initialization (1, 2, 3, ...)
      for (int j = 0; j < N; j++) {
         A[i][j] = (i + 1) * (j + 1); // Matrix initialization
      }
   }

   // Print matrix A
   printf("Matrix A:\n");
   for (int i = 0; i < N; i++) {
      for (int j = 0; j < N; j++) {
         printf("%d ", A[i][j]);
      }
      printf("\n");
   }

   // Print vector x
   printf("Vector x:\n");
   for (int i = 0; i < N; i++) {
      printf("%d ", x[i]);
   }
   printf("\n");
```

```
    // Start timing
    double start_time = omp_get_wtime();

    // Perform matrix-vector multiplication in parallel
    #pragma omp parallel for
    for (int i = 0; i < N; i++) {
        y[i] = 0; // Initialize the result vector
        for (int j = 0; j < N; j++) {
            y[i] += A[i][j] * x[j];
        }
    }

    // End timing
    double end_time = omp_get_wtime();
    printf("Time taken for matrix-vector multiplication: %f seconds\n", end_time - start_time);

    // Print the result vector
    printf("Result vector y:\n");
    for (int i = 0; i < N; i++) {
        printf("y[%d] = %d\n", i, y[i]);
    }

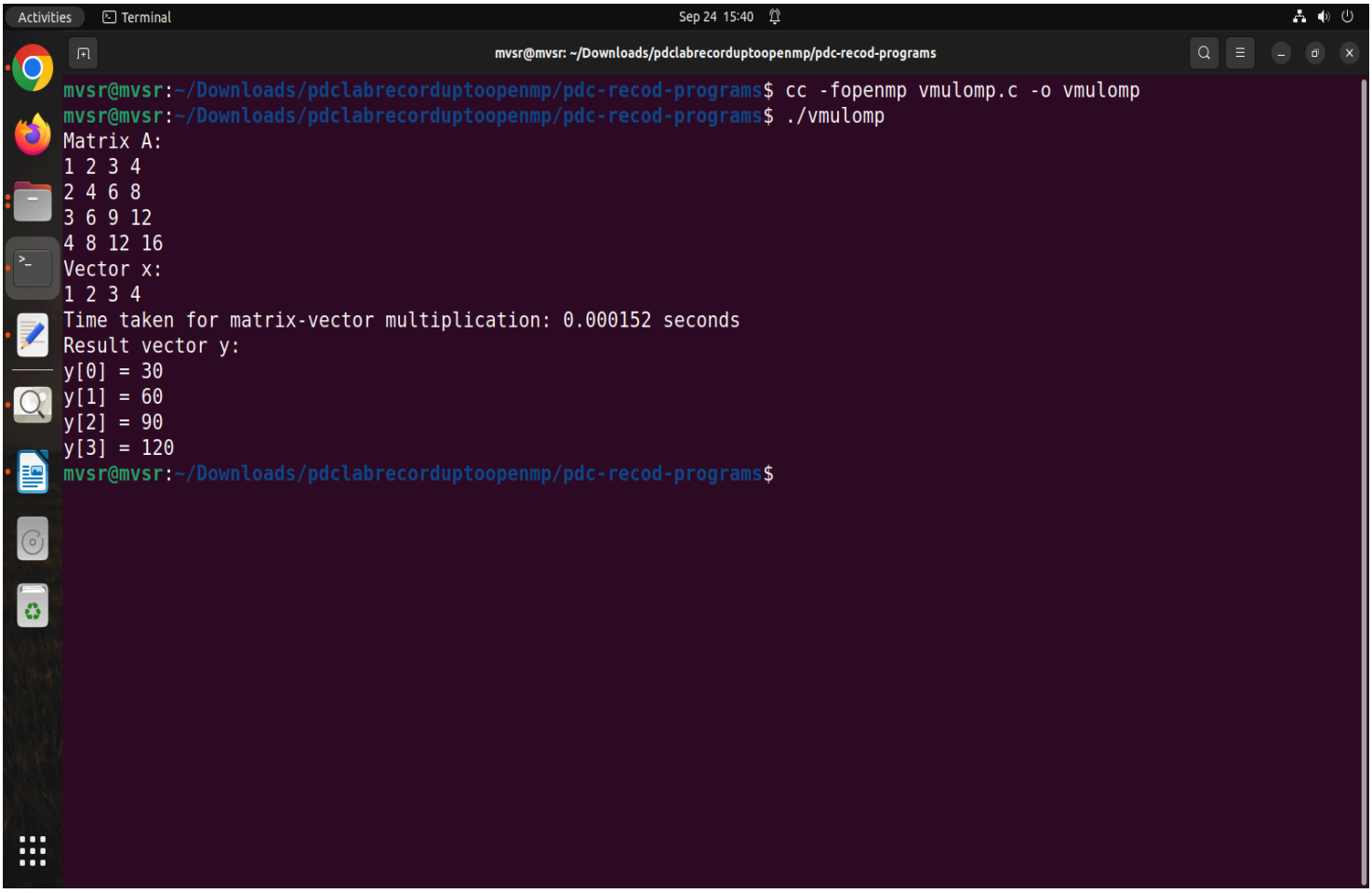
    return 0;
}



// Compilation: To compile this program with OpenMP, use the following command:
// cc -fopenmp vmulomp.c -o vmulomp


// EXECUTION
// ./vmulmp
```

output



OpenMP Directives

THEORY

OpenMP Directives Format

OpenMP directives for C/C++ are specified with the pragma preprocessing directive.

#pragma omp directive-name [clause[[,] clause]. . .] new-line

Where:

- **#pragma omp:** Required for all OpenMP C/C++ directives.

- **directive-name:** A valid OpenMP directive. Must appear after the pragma and before any clauses.

- **[clause, ...]:** Optional, Clauses can be in any order, and repeated as necessary unless otherwise restricted.

- **Newline:** Required, Precedes the structured block which is enclosed by this directive.

General Rules:

- Case sensitive
- Directives follow conventions of the C/C++ standards for compiler directives
- Only one directive-name may be specified per directive
- Each directive applies to at most one succeeding statement, which must be a structured block.
- Long directive lines can be "continued" on succeeding lines by escaping the newline character with a backslash ("\") at the end of a directive line.

OpenMP Directives or Constructs

• Parallel Construct

• Work-Sharing Constructs

- Loop Construct
- Sections Construct

- Single Construct

• Data-Sharing, No Wait, and Schedule Clauses

• Barrier Construct

• Critical Construct

• Atomic Construct

• Locks

• Master Construct

Directive Scoping Static

(Lexical) Extent:

- The code textually enclosed between the beginning and the end of a structured block following a directive.
- The static extent of a directives does not span multiple routines or code files

Orphaned Directive:

- An OpenMP directive that appears independently from another enclosing directive is said to be an orphaned directive. It exists outside of another directive's static (lexical) extent.
- Will span routines and possibly code files

Dynamic Extent:

- The dynamic extent of a directive includes both its static (lexical) extent and the extents of its orphaned directives.

Parallel Construct

This construct is used to specify the computations that should be executed in parallel. Parts of the program that are not enclosed by a parallel construct will be executed serially. When a thread encounters this construct, a team of threads is created to execute the associated parallel region, which is the code dynamically contained within the parallel construct. But although this construct ensures that computations are performed in parallel, it does not distribute the work of the region among the threads in a team. In fact, if the programmer does not use the appropriate syntax to specify this action, the work will be replicated. At the end of a parallel region, there is an implied *barrier* that forces all threads to wait until the work inside the region has been completed. Only the initial thread continues execution after the end of the parallel region.

The thread that encounters the parallel construct becomes the *master* of the new team. Each thread in the team is assigned a unique thread number (also referred to as the "thread id") to identify it. They range from zero (for the master thread) up to one less than the number of threads within the team, and they can be accessed by the programmer. Although the parallel region is executed by all threads in the team, each thread is allowed to follow a different path of execution.

Format

```
#pragma omp parallel [clause ...] ……. newline
                        if (scalar_expression) private
                        (list)
                        shared (list)
                        default (shared | none)
                        firstprivate (list) reduction
                        (operator: list) copyin (list)
                        num_threads (integer-expression)
{
 structured_block
            }
```

Example of a parallel region

```
    #include <omp.h> main()
            {
                      #pragma omp parallel
               {
                    printf("The parallel region is executed by
                    thread %d\n",
                    omp_get_thread_num());

                 } /*-- End of parallel region --*/
                       }/*-- End of Main Program --*/
```

Here, the OpenMP library function **omp_get_thread_num()** is used to obtain the number of each thread executing the parallel region. Each thread will execute all code in the parallel region, so that we should expect each to perform the print statement.. Note that one cannot make any assumptions about the order in which the threads will execute the printf statement. When the code is run again, the order of execution could be different.

Possible output of the code with four threads.

> The parallel region is executed by thread 0 The parallel region is executed by thread 3 The parallel region is executed by thread 2 The parallel region is executed by thread 1.

Clauses supported by the parallel construct

- **if***(scalar-expression)*
- **num threads***(integer-expression)*
- **private***(list)*
- **firstprivate***(list)*
- **shared***(list)*
- **default(none|shared)**
- **copyin***(list)*
- **reduction***(operator:list)*

Details and usage of clauses are disused in lab session B.4

Key Points:

- A program that branches into or out of a parallel region is nonconforming. In other words, if a program does so, then it is *illegal*, and the behavior is undefined.
- A program must not depend on any ordering of the evaluations of the clauses of the parallel directive or on any side effects of the evaluations of the clauses.
- At most one if clause can appear on the directive.
- At most one num_threads clause can appear on the directive. The expression for the clause must evaluate to a positive integer value.

Determining the Number of Threads for a parallel Region

When execution encounters a parallel directive, the value of the if clause or num_threads clause (if any) on the directive, the current parallel context, and the values of the nthreads-var, dyn-var, thread-limit-var, max-active-level-var, and nest-var ICVs are used to determine the number of threads to use in the region.

Note that using a variable in an if or num_threads clause expression of a parallel construct causes an implicit reference to the variable in all enclosing constructs. The if clause expression and the num_threads clause expression are evaluated in the context outside of the parallel construct, and no ordering of those evaluations is specified. It is also unspecified whether, in what order, or how many times any side-effects of the evaluation of the num_threads or if clause expressions occur.

Example: use of num_threads Clause

The following example demonstrates the num_threads clause. The parallel region is executed

with a maximum of 10 threads.

```
#include <omp.h> main()
      {
                ...
```

```
                #pragma omp parallel num_threads(10)
                {
                    ... parallel region ...
                }
        }
```

Specifying a Fixed Number of Threads

Some programs rely on a fixed, pre-specified number of threads to execute correctly. Because the default setting for the dynamic adjustment of the number of threads is implementation- defined, such programs can choose to turn off the dynamic threads capability and set the number of threads explicitly to ensure portability. The following example shows how to do this using omp_set_dynamic and omp_set_num_threads

Example:

```
#include <omp.h> main()
     {
         omp_set_dynamic(0);
          omp_set_num_threads(16);

             #pragma omp parallel num_threads(10)
              {


                                                                    }
    ... parallel region ...
}
```

OBJECT

Clauses in Loop Construct

THEORY

Introduction

The OpenMP Data Scope Attribute Clauses are used to explicitly define how variables should be scoped.

Data Scope Attribute Clauses are used in conjunction with several directives (PARALLEL, DO/for, and SECTIONS) to control the scoping of enclosed variables.

These constructs provide the ability to control the data environment during execution of parallel constructs.

- They define how and which data variables in the serial section of the program are transferred to the parallel sections of the program (and back)
- They define which variables will be visible to all threads in the parallel sections and which variables will be privately allocated to all threads.

List of Clauses

- PRIVATE
- FIRSTPRIVATE
- LASTPRIVATE
- SHARED
- DEFAULT
- REDUCTION
- COPYIN

PRIVATE Clause

The PRIVATE clause declares variables in its list to be private to each thread.

Format: PRIVATE (list)

Notes:

- PRIVATE variables behave as follows:

- A new object of the same type is declared once for each thread in the team
- All references to the original object are replaced with references to the new object
- Variables declared PRIVATE are uninitialized for each thread

Example of the private clause – Each thread has a local copy of variables i and a.

```
#pragma omp parallel for private(i,a) for
      (i=0; i<n; i++)
      {
            a = i+1;
            printf("Thread %d has a value of a = %d for i =
            %d\n", omp_get_thread_num(),a,i);
      } /*-- End of parallel for --*/
```

SHARED Clause

The SHARED clause declares variables in its list to be shared among all threads in the team.

Format: SHARED *(list)*

Notes:

- A shared variable exists in only one memory location and all threads can read or write to that address
- It is the programmer's responsibility to ensure that multiple threads properly access SHARED variables (such as via CRITICAL sections)

Example of the shared clause – All threads can read from and write to vector a.

```
#pragma omp parallel for shared(a) for
      (i=0; i<n; i++)
      {
            a[i] += i;
      } /*-- End of parallel for --*/
```

DEFAULT Clause

The DEFAULT clause allows the user to specify a default PRIVATE, SHARED, or NONE scope for all variables in the lexical extent of any parallel region.
The default clause is used to give variables a default data-sharing attribute. Its usage is straightforward. For example, default (shared) assigns the shared attribute to all variables

referenced in the construct. This clause is most often used to define the data-sharing attribute of the majority of the variables in a parallel region. Only the exceptions need to be explicitly listed. If default(none) is specified instead, the programmer is forced to specify a data-sharing attribute for each variable in the construct. Although variables with a predetermined data- sharing attribute need not be listed in one of the clauses, it is strongly recommend that the attribute be explicitly specified for *all* variables in the construct.

Format: DEFAULT (SHARED | NONE)

Notes:

- Specific variables can be exempted from the default using the PRIVATE, SHARED, FIRSTPRIVATE, LASTPRIVATE, and REDUCTION clauses
- The C/C++ OpenMP specification does not include "private" as a possible default. However, actual implementations may provide this option.
- Only one DEFAULT clause can be specified on a PARALLEL directive

Example of the Deafulat clause: all variables to be shared, with the exception of a, b, and c.

```
#pragma omp for default(shared) private(a,b,c),
```

FIRSTPRIVATE Clause

The FIRSTPRIVATE clause combines the behavior of the PRIVATE clause with automatic initialization of the variables in its list.

Format: FIRSTPRIVATE *(LIST)*

Notes:

- Listed variables are initialized according to the value of their original objects prior to entry into the parallel or work-sharing construct

Example using the firstprivate clause – Each thread has a pre-initialized copy of variable indx. This variable is still private, so threads can update it individually.

```
for(i=0; i<vlen; i++) a[i] = -i-1; indx =
4;
{
#pragma omp parallel default(none) firstprivate(indx)
private(i,TID) shared(n,a)

      {
```

```
TID = omp_get_thread_num(); indx
+= n*TID;
```

```
            for(i=indx; i<indx+n; i++)
            a[i] = TID + 1;
        }
    }
    printf("After the parallel region:\n"); for
    (i=0; i<vlen; i++)
    printf("a[%d] = %d\n",i,a[i]);
```

LASTPRIVATE Clause

The LASTPRIVATE clause combines the behavior of the PRIVATE clause with a copy from the last loop iteration or section to the original variable object.

Format: LASTPRIVATE *(LIST)*

Notes:

- The value copied back into the original variable object is obtained from the last (sequentially) iteration or section of the enclosing construct.
- It ensures that the last value of a data object listed is accessible after the corresponding construct has completed execution

Example of the lastprivate clause – This clause makes the sequentially last value of variable a accessible outside the parallel loop.

```
    #pragma omp parallel for private(i) lastprivate(a) for
        (i=0; i<n; i++)
        {
            a = i+1;
            printf("Thread %d has a value of a = %d for i =
            %d\n", omp_get_thread_num(),a,i);
        } /*-- End of parallel for --*/

          printf("Value of a after parallel for: a =
%d\n",a);
```

COPYIN Clause

The COPYIN clause provides a means for assigning the same value to THREADPRIVATE variables for all threads in the team.

Format: COPYIN *(LIST)*

Notes:

- List contains the names of variables to copy. The master thread variable is used as the copy source. The team threads are initialized with its value upon entry into the parallel construct.

REDUCTION Clause

The REDUCTION clause performs a reduction on the variables that appear in its list. A private copy for each list variable is created for each thread. At the end of the reduction, the reduction variable is applied to all private copies of the shared variable, and the final result is written to the global shared variable.

Format: REDUCTION *(OPERATOR: LIST)*

Notes:

- Variables in the list must be named scalar variables. They can not be array or structure type variables. They must also be declared SHARED in the enclosing context.
- Reduction operations may not be associative for real numbers.
- The REDUCTION clause is intended to be used on a region or work-sharing construct in which the reduction variable is used only in statements which have one of following forms:

C / C++
x = x op expr **x = expr op x** (except subtraction) **x binop = expr x++** **++x** **x--** **--x**
x is a scalar variable in the list ***expr*** is a scalar expression that does not reference ***x*** ***op*** is not overloaded, and is one of +, *, -, /, &, ^, |, &&, || ***binop*** is not overloaded, and is one of +, *, -, /, &, ^, |

Example of REDUCTION - Vector Dot Product. Iterations of the parallel loop will be distributed in equal sized blocks to each thread in the team (SCHEDULE STATIC). At the end of the parallel loop construct, all threads will add their values of "result" to update the master thread's global copy.

```
#include <omp.h>
main ()
{
      int   i, n, chunk;
      float a[100], b[100], result; n =
      100;
```

```
chunk = 10;
result = 0.0;
```

```
      for (i=0; i < n; i++)
        {
        a[i] = i * 1.0;
        b[i] = i * 2.0;
        }
      #pragma omp parallel for    default(shared) private(i)
      schedule(static,chunk) reduction(+:result)
        {
           for (i=0; i < n; i++)
           result = result + (a[i] * b[i]);

      printf("Final result= %f\n",result);
      }
}
```

SCHEDULE:

Describes how iterations of the loop are divided among the threads in the team. The default schedule is implementation dependent.

STATIC

Loop iterations are divided into pieces of size *chunk* and then statically assigned to threads. If chunk is not specified, the iterations are evenly (if possible) divided contiguously among the threads.

DYNAMIC

Loop iterations are divided into pieces of size *chunk*, and dynamically scheduled among the threads; when a thread finishes one chunk, it is dynamically assigned another. The default chunk size is 1.

GUIDED

For a chunk size of 1, the size of each chunk is proportional to the number of unassigned iterations divided by the number of threads, decreasing to 1. For a chunk size with value k (greater than 1), the size of each chunk is determined in the same way with the restriction that the chunks do not contain fewer than k iterations (except for the last chunk to be assigned, which may have fewer than k iterations). The default chunk size is 1.

Nowait Clause

The nowait clause allows the programmer to fine-tune a program's performance. When we introduced the work-sharing constructs, we mentioned that there is an implicit barrier at the end of them. This clause overrides that feature of OpenMP; in other words, if it is added to a construct, the barrier at the end of the associated construct will be suppressed. When threads reach the end of the construct, they will immediately proceed to perform other work. Note,

however, that the barrier at the end of a parallel region cannot be suppressed.

Example of the nowait clause in C/C++ – The clause ensures that there is no barrier at the end of the loop.

```
#pragma omp for nowait for
     (i=0; i<n; i++)
     {
     ............
     }
```

Clauses / Directives Summary

Table 11.1 Comparative Analysis for a set of instruction

Clause	Directive					
	PARALLEL	DO/for	SECTIONS	SINGLE	PARALLEL DO/for	PARALLEL SECTIONS
IF	●				●	●
PRIVATE	●	●	●	●	●	●
SHARED	●	●			●	●
DEFAULT	●				●	●
FIRSTPRIVATE	●	●	●	●	●	●
LASTPRIVATE		●	●		●	●
REDUCTION	●	●	●		●	●
COPYIN	●				●	●
SCHEDULE		●			●	
ORDERED		●			●	
NOWAIT		●	●	●		

- The following OpenMP directives do not accept clauses:
    - MASTER
    - CRITICAL
    - BARRIER
    - ATOMIC
    - FLUSH

- ORDERED
- THREADPRIVATE

- ***Implementations may (and do) differ from the standard in which clauses are supported by each directive.***

3. Program for Parallel reduction to calculate the sum of the array elements using openmp

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

int main() {
    int size;

    // Take the size of the array as input
    printf("Enter the size of the array: ");
    scanf("%d", &size);

    // Dynamically allocate memory for the array
    float *arr = (float*) malloc(size * sizeof(float));

    // Input values for the array
    printf("Enter the values for the array:\n");
    for (int i = 0; i < size; i++) {
        scanf("%f", &arr[i]);
    }

    float sum = 0.0;

    // Parallel reduction to calculate the sum of the array elements
    #pragma omp parallel for reduction(+:sum)
    for (int i = 0; i < size; i++) {
        sum += arr[i];
    }

    // Output the result
    printf("Sum of the array elements: %.2f\n", sum);

    // Free the dynamically allocated memory
    free(arr);

    return 0;
}
```

//Compilation: To compile this program with OpenMP support, use the following command:
//cc -fopenmp predompasum.c -o predompasum

//execution:
//./predompasum

OUTPUT:

